ENGINEERING CASE STUDY

Serverless File Processing System

An event-driven file processing architecture on AWS

AWS CLOUD · SERVERLESS · DEVOPS

AUTHOR
Sumanth R Malagar

COMPLETION
18 October 2026

DOCUMENT VERSION
1.0

REGION
ap-south-1

TECHNOLOGIES
Amazon S3 · AWS Lambda · Amazon DynamoDB · Amazon SQS · Amazon CloudWatch · AWS IAM · Python 3.13

14	24	25	135
CHAPTERS	FIGURES	TABLES	AUTOMATED TESTS

> *Everything here comes from the project repo, the AWS Console screenshots and the test run.*

Executive Summary

The Serverless File Processing System (SFPS) is an event-driven AWS pipeline that validates uploaded files, extracts metadata, computes a SHA-256 checksum, and records the outcome of every object version in Amazon DynamoDB. A file placed under the incoming/ prefix of a private, versioned Amazon S3 bucket asynchronously invokes a single AWS Lambda function. The function validates the extension, size, content signature, and decodability of the file, extracts type-specific metadata, hashes the original bytes, and writes a deterministic processing record. Structured JSON logs go to Amazon CloudWatch Logs. Failures that exhaust asynchronous retries are delivered to an Amazon SQS queue.

The project was built as a complete V1 rather than an architecture sketch. The repository contains the Lambda source, a pure validation layer, image and text processors, a SHA-256 helper, a pytest suite with moto-mocked AWS services, a requirements lock, and a manual AWS deployment guide. The function was deployed to the ap-south-1 region on Python 3.13 with the arm64 architecture, 512 MB of memory, and a 30-second timeout. The deployment package was built on Windows with Linux arm64 wheels, uploaded as a ZIP archive, and verified for layout and dependency versions.

The design concentrates on three properties that separate a working demo from a dependable processing pipeline. First, every read is pinned to the object version carried by the S3 event, so the bytes that are validated, hashed, and measured are the bytes identified by the record. Second, the DynamoDB write protocol uses conditional writes and a deterministic processing_id derived from the bucket, object key, and version ID, so duplicate deliveries cannot create a second logical record or overwrite a completed result. Third, failures are classified. Permanent validation failures become FAILED records and do not trigger retries; infrastructure failures are logged with sanitized frames, re-raised, retried by Lambda, and delivered to SQS when the retries are exhausted.

The evidence collected on 18 October 2026 shows the deployed system processing the planned acceptance inputs. The DynamoDB table held 25 records at the time of the scan: 10 COMPLETED and 15 FAILED. Those 25 records are not 25 independent test scenarios. They include two versions of the same sample image, seven zero-byte folder-marker records, and the positive and negative case-study fixtures. The captured SQS message shows the expected invocation envelope with condition RetriesExhausted, approximateInvokeCount 3, the original event under requestPayload, and a sanitized InfrastructureError in responsePayload. One successful PNG invocation was recorded at 89 ms of application processing, 150.13 ms of Lambda execution, and 117 MB of peak memory against a 512 MB configuration.

The automated suite was re-run for this case study: 135 tests passed in 48.66 seconds, Ruff reported no issues, and pip check found no broken requirements. The deployment archive contained 2,358 entries, measured 22,955,647 bytes compressed and 45,828,329 bytes uncompressed, placed handler.py at the archive root, used forward-slash entry names, and included eight aarch64 Pillow extension libraries with all eight runtime dependency versions matching requirements.txt.

The case study does not claim production-grade security, fault tolerance, or general performance. It does not claim that the 25 MiB memory and timeout boundary was measured on the deployed function, that every per-file error code is legible in the captured console views, or that any AWS cost figure is known beyond the pay-per-use model. Those gaps are listed with the evidence inventory and the outstanding tests. The value of the project lies in the combination of a working deployed pipeline, a failure model that distinguishes expected rejections from operational faults, and a documented trail that makes the claims checkable.

Project Highlights

What the project demonstrates

- Event-driven flow from a private, versioned S3 bucket to a single Python 3.13 Lambda function and a DynamoDB processing table.
- Validation order that rejects unsupported extensions before download, then checks size, content signature, image decodability, or strict UTF-8 text.
- One deterministic DynamoDB record per S3 object version, protected by conditional writes and a skip-only-COMPLETED idempotency rule.
- Metadata extraction for JPEG, PNG, and text, plus SHA-256 over the version-pinned original bytes.
- Permanent validation failures recorded as FAILED without retries; infrastructure failures re-raised through Lambda's asynchronous retry path.
- Structured JSON logs with allowlisted error messages and sanitized stack frames; 30-day log retention.
- On-failure SQS destination confirmed by a captured invocation envelope with RetriesExhausted and approximateInvokeCount 3.
- 135 automated tests, a clean Ruff run, and a verified Linux arm64 deployment archive built from Windows.

Table of Contents

Executive Summary 2
Project Highlights 2
Chapter 1. Introduction 6
Chapter 2. Problem Statement and Requirements 8
Chapter 3. Technology Stack 11
Chapter 4. System Architecture 13
Chapter 5. Application Design and Processing Logic 16
Chapter 6. AWS Infrastructure Implementation 20
Chapter 7. Deployment and Packaging 25
Chapter 8. Security and Reliability 26
Chapter 9. Testing Strategy and Test Execution 30
Chapter 10. Results and Performance Analysis 37
Chapter 11. Challenges and Solutions 40
Chapter 12. Cost and Operational Considerations 41
Chapter 13. Limitations and Future Enhancements 43
Chapter 14. Conclusion and Lessons Learned 44
Appendix A. Configuration Reference 45
Appendix B. Selected JSON and Log Shapes 45
Appendix C. Deployment Commands 46
Appendix D. Architecture References 46
Appendix E. Screenshot Evidence Index 47
Appendix F. Glossary 47

List of Figures

Figures used in this document

Figure	Title
Figure 4.1	Original SFPS event-driven architecture diagram, drawn from the implemented code and deployed configuration.
Figure 4.2	Processing sequence for one S3 object version, from event delivery through the terminal DynamoDB write.
Figure 4.3	Deployed Lambda function overview showing the S3 trigger on the left and the SQS destination on the right.
Figure 6.1	S3 bucket object view showing the incoming/ prefix at the bucket root.
Figure 6.2	Objects under incoming/, including sample.jpg, sample.png, sample.pdf, and images.jpg.
Figure 6.3	DynamoDB table browser for sfps-processing with the Scan view selected.
Figure 6.4	Lambda function list showing sfps-processor as a Zip package on Python 3.13.
Figure 6.5	Lambda general configuration showing 512 MB of memory and a 30-second timeout.
Figure 6.6	Lambda environment variables for the deployed function.
Figure 6.7	S3 trigger details showing ObjectCreated:* events, the incoming/ prefix, and the source account.
Figure 6.8	CloudWatch log group for the processing function.
Figure 6.9	Recent CloudWatch log streams for the processing function.
Figure 8.1	IAM execution role page captured for the deployed function.
Figure 8.2	Lambda asynchronous invocation destination configured for On failure to the SQS queue.
Figure 8.3	SQS queue with one received message after an intentionally triggered infrastructure failure.
Figure 9.1	S3 test-fixture structure under incoming/case-study-tests/ with separate positive and negative folders.
Figure 9.2	Negative test fixtures, including 08-unsupported.csv, 09-invalid-utf8.txt, and 10-fake-jpeg.jpg.
Figure 9.3	Positive test fixtures, including 04-multiline.txt, 05-unicode.txt, and the nested/ folder.
Figure 9.4	DynamoDB scan summary showing 25 items returned, including completed sample records and failed versions.
Figure 9.5	DynamoDB rows for the additional positive and negative fixtures, including 01-landscape.jpg, 11-corrupt-png.png, and 05-unicode.txt.
Figure 9.6	Additional DynamoDB rows including 03-square.png, sample.txt, 02-portrait.jpeg, 12-over-pixel-limit.png, and 08-unsupported.csv.
Figure 9.7	Final DynamoDB rows including 09-invalid-utf8.txt, 06-nested-image.jpg, 10-fake-jpeg.jpg, and sample.jpg.

Figure	Title
Figure 9.8	CloudWatch log events for a permanent validation failure, showing platform lines and structured application lines.
Figure 10.1	CloudWatch log events showing two failed invocations and their REPORT durations.

List of Tables

Tables used in this document

Table	Title
Table 0.1	Abbreviations and terms used in this document.
Table 1.1	Evidence status categories used throughout this document.
Table 2.1	Functional requirements and implementation status.
Table 2.2	Nonfunctional requirements and current evidence.
Table 2.3	Supported file types, validation rules, and configured limits.
Table 2.4	Error codes stored in the DynamoDB error map.
Table 3.1	Technology responsibilities and interactions.
Table 4.1	Failure classes and their behavior.
Table 5.1	Lambda environment variables.
Table 5.2	Validation order and failure mapping.
Table 5.3	DynamoDB record attributes.
Table 5.4	Processing states and transitions.
Table 6.1	Documented IAM policy statements.
Table 7.1	Independent verification of the deployment archive.
Table 8.1	Security controls and evidence status.
Table 9.1	Collected test distribution.
Table 9.2	Composition of the 25-item DynamoDB scan.
Table 9.3	Deployed functional and validation test matrix.
Table 9.4	Automated, regression, and boundary test matrix.
Table 10.1	Observed performance values and their scope.
Table 12.1	Cost drivers and qualitative controls.
Table 13.1	Limitations and prioritized enhancements.
Table A.1	Configuration reference for the deployed V1 environment.
Table E.1	Figure, source, and evidence supported.
Table F.1	Glossary of project terms.

Abbreviations and Terminology

Table 0.1. *Abbreviations and terms used in this document.*

Term	Meaning
ADR	Architecture decision record
ARM64	64-bit Arm processor architecture used by the Lambda function
AWS	Amazon Web Services
boto3 / botocore	AWS SDK for Python and its low-level core library
CI/CD	Continuous integration and continuous delivery
CLI	Command-line interface
CloudWatch	Amazon CloudWatch, used here for logs
DLQ	Dead-letter queue; SFPS uses a Lambda on-failure destination instead
DynamoDB	Amazon DynamoDB, the processing-state store
IAM	AWS Identity and Access Management
IaC	Infrastructure as code
JSON	JavaScript Object Notation
Lambda	AWS Lambda, the compute service for the processing function
MiB	Mebibyte; 1 MiB = 1,048,576 bytes
PRD	Product requirements document
SFPS	Serverless File Processing System
SHA-256	Secure Hash Algorithm with a 256-bit digest
SQS	Amazon Simple Queue Service
SSE-S3	Server-side encryption with Amazon S3 managed keys
SSE-SQS	Server-side encryption with Amazon SQS managed keys
TLS	Transport Layer Security
UTF-8	Unicode Transformation Format, 8-bit

Term	Meaning
V1	The first implemented release documented here
ZIP	The deployment archive format used for the Lambda package

Chapter 1. Introduction

1.1 Background

File processing is a routine backend task with a deceptive amount of detail. A system receives a file, decides whether it is acceptable, extracts information from it, stores the result, and tells an operator what happened. In a small prototype, a single script can do all of that. In an operational system, the same script has to handle duplicate deliveries, partial failures, large inputs, malformed content, security boundaries, retries, and the need to explain later why a particular file produced a particular record.

Traditional server-based processing puts a long-running worker in front of that problem. The worker polls a queue or watches a directory, keeps libraries loaded, and writes to a database. This model is familiar and can be efficient at sustained load. It also asks the team to manage capacity, patching, idle time, and failure recovery for a process that may receive only a few files an hour. A serverless model replaces the idle worker with an event source and a function that runs only when work arrives. The operational surface shrinks, but the correctness surface does not. The function still needs idempotency, bounded resource use, and a clear failure model.

SFPS explores that trade-off in a concrete AWS deployment. The project uses Amazon S3 as the source of truth for uploaded bytes, AWS Lambda as the processing unit, Amazon DynamoDB as the processing ledger, Amazon CloudWatch Logs for diagnostics, Amazon SQS for exhausted failures, and AWS IAM for the execution boundary. The implementation is deliberately narrow: four file extensions, one function, one table, one queue, and a manual deployment. The narrow scope makes it possible to examine each decision and each failure path in detail.

1.2 Motivation

The project began with two practical questions. Could a file-processing pipeline be built and operated on AWS without a permanently running server? And could the resulting system be defended with evidence rather than assertions? The first question is an architecture exercise. The second is an engineering discipline exercise. A deployed function that appears to work in a console screenshot is not the same as a system whose validation order, idempotency rules, error taxonomy, and deployment package have been checked.

The work therefore treated the repository, the deployment, and the evidence as one artifact. Source code and tests describe intended behavior. Console captures and a captured SQS envelope describe what the deployed resources actually did. The case study keeps those sources separate and says which one supports each claim. This is slower than writing a feature list, but it produces a document that a reviewer can audit.

1.3 Purpose and objectives

The project pursued six objectives:

1. Build an event-driven pipeline in which an S3 upload automatically invokes Lambda and produces a durable processing record.
2. Validate file type by extension, size, content signature, and decodability rather than trusting the file name.
3. Extract useful metadata for supported image and text formats and compute a SHA-256 checksum over the original bytes.
4. Make processing idempotent across duplicate S3 deliveries and safe under concurrent invocations.
5. Separate permanent validation failures from transient infrastructure failures so that each follows the correct retry path.
6. Deploy the system manually, document the steps, and collect evidence that distinguishes verified behavior from intended behavior.

1.4 Scope

V1 covers a single AWS region and a single processing function. The supported inputs are .jpg, .jpeg, .png, and .txt. Extensions are case-insensitive. Files may be uploaded directly under incoming/ or under nested prefixes such as incoming/2026/photo.jpg or incoming/users/demo/file.txt. The S3 notification uses the incoming/ prefix and every s3:ObjectCreated:* event, with no suffix filter. Unsupported files therefore reach Lambda and are recorded as FAILED instead of being silently ignored.

The implemented resource set is Amazon S3, AWS Lambda, Amazon DynamoDB, Amazon SQS, Amazon CloudWatch Logs, and AWS IAM. Deployment is manual through the AWS Management Console, with AWS CLI commands for package upload and selected verification steps. There is no API Gateway, web frontend, Step Functions state machine, SNS topic, malware scanner, image resizer, or content-deduplication service. There is no infrastructure as code, CI/CD pipeline, or automated provisioning in V1. Those items are listed as future work, not as hidden features.

1.5 Intended users

The document serves four audiences. A hiring manager or technical recruiter can read the executive summary, the highlights, and the results chapter to understand the scope and the evidence. A cloud or backend engineer can read the architecture, design, infrastructure, and testing chapters to evaluate the implementation. A reviewer can use the evidence inventory and the appendices to check specific claims. A future maintainer can use the configuration reference and the deployment commands to reproduce the environment.

1.6 The cost of manual and server-based processing

Manual file handling does not scale because every upload depends on a person remembering the same sequence of checks. The checks drift, the results are not recorded consistently, and there is no reliable audit trail. A shared server improves consistency but introduces a new set of problems: the worker must be provisioned for peak load, patched for library and OS vulnerabilities, monitored for memory growth, and restarted after crashes. If the worker processes files sequentially, one bad input can block the queue. If it processes them in parallel, the database writes need concurrency control. Idle capacity is paid for even when no files arrive.

The event-driven model addresses the capacity and idle-cost problems, but it introduces at-least-once delivery and a harder failure boundary. S3 event notifications can be delivered more than once. Lambda asynchronous invocations can retry. A timeout or an unhandled exception can leave a record in an intermediate state. The design work moves from managing a server to managing identity, retries, state transitions, and evidence. SFPS treats those concerns as the core of the project.

Table 1.1. *Evidence status categories used throughout this document.*

Status	Meaning	Example in this project
Implemented and verified	The behavior exists in source and has direct execution evidence from the repository test suite or the deployed environment.	The 135-test pytest suite; the captured SQS invocation envelope; the 512 MB and 30-second Lambda configuration shown in the console.
Implemented, not independently verified	The behavior exists in source and is documented, but the available evidence does not prove it in the deployed environment.	The exact IAM policy attachment; the 30-day CloudWatch retention value; the 25 MiB memory and timeout boundary on the deployed function.
Planned or documented only	The requirement or enhancement appears in the PRD or planning documents but is not part of the V1 implementation.	Infrastructure as code, CI/CD, CloudWatch alarms, automated failure replay.
Future enhancement	A sensible extension that follows from the current design but was not built.	Additional file formats, malware scanning, load testing, cost dashboards.
Not claimed	A property that the evidence does not support and that this document does not assert.	Production-grade fault tolerance, a general performance benchmark, or an actual AWS bill.

1.7 Document conventions

Chapters are numbered sequentially and use a decimal scheme for subsections. Figures and tables are numbered by chapter. Code excerpts are reproduced from the repository with minor whitespace adjustments for page width; they are not paraphrased. The AWS account identifier is written as <account-id> in prose and in the redacted SQS excerpt. The console screenshots show the project account identifier as part of resource names and ARNs; no access keys, secret keys, session tokens, or passwords appear in any capture.

The document uses the terms record, processing record, and item interchangeably for a DynamoDB item in the sfps-processing table. A validation failure means a permanent rejection of the input, such as an unsupported extension or corrupt image. An infrastructure failure means an AWS service error, an unexpected exception, or a configuration error that should be retried or investigated. A FAILED record is not the same thing as a failed Lambda invocation.

Chapter 2. Problem Statement and Requirements

2.1 Problem statement

The system must accept files without a permanently running worker, decide whether each file is acceptable, extract metadata that is useful to a downstream consumer, and preserve a record of the outcome. The record must identify the exact S3 object version that was processed, including its version ID, rather than the object key alone. Duplicate notifications for the same version must not create a second logical result or overwrite a completed one. A bad file must fail in a way that is visible to an operator without causing an endless retry loop. A genuine infrastructure fault must be retried and, if it persists, must leave the original event somewhere an operator can inspect.

The problem has a security dimension as well. Uploaded files are untrusted input. The function must not execute them, must not trust their extensions, and must not write raw exception text or file contents into logs or the database. The bucket must remain private, encrypted, and reachable only over TLS. The execution role must be limited to the specific bucket prefix, table, queue, and log group used by the function.

2.2 Functional requirements

The functional requirements below are derived from the PRD and checked against the implementation. The status column distinguishes requirements that are implemented and verified from those that are implemented but lack direct deployment evidence.

Table 2.1. *Functional requirements and implementation status.*

ID	Requirement	Implementation evidence	Status
FR-01	An object created under incoming/ automatically invokes the processing function.	S3 trigger with s3:ObjectCreated:* and prefix incoming/ (Figure 6.7); Lambda overview (Figure 4.3).	Implemented, verified in console
FR-02	Nested keys under incoming/ are processed.	Handler prefix check and extension parsing; 06-nested-image.jpg record; positive/nested/ folder in Figure 9.3.	Implemented, observed
FR-03	Object size is read from a version-pinned HeadObject before content download.	handler._head_object_size passes VersionId; automated version-pinning test.	Implemented and verified
FR-04	Zero-byte objects are rejected.	ValidationError ZERO_BYTE after HeadObject; seven folder-marker records show FAILED.	Implemented and verified
FR-05	Objects above MAX_FILE_SIZE_BYTES are rejected.	Size check before GetObject; automated 25 MiB and 25 MiB + 1 byte tests.	Implemented and verified
FR-06	Only jpg, jpeg, png, and txt extensions are accepted, case-insensitively.	extract_extension and validate_extension; unit tests; deployed sample.pdf and 08-unsupported.csv failures.	Implemented and verified
FR-07	Image content must match its extension signature and decode fully.	JPEG/PNG signature check, Pillow verify, reopen and decode every frame.	Implemented and verified
FR-08	Image metadata stores width, height, and detected format.	processors.image.extract_metadata; completed image records in DynamoDB.	Implemented and verified
FR-09	Text must be strict UTF-8; BOM is stripped; metadata stores characters, words, and lines.	validation.decode_text and processors.text.extract_metadata; unit tests; 05-unicode.txt completed.	Implemented and verified
FR-10	SHA-256 is computed over the original version-pinned bytes.	checksum.sha256_hex; automated known-digest and version-pinning tests.	Implemented and verified
FR-11	Each object version gets exactly one deterministic processing record.	processing_id from canonical JSON of bucket, key, and version; conditional PutItem.	Implemented and verified
FR-12	Duplicate delivery of a COMPLETED version is skipped without changing the record.	Conditional claim and terminal UpdateItem; duplicate test.	Implemented and verified
FR-13	Permanent validation failures become FAILED and do not trigger retries.	ValidationError path writes FAILED and returns; deployed failed records.	Implemented and observed
FR-14	Infrastructure failures are logged, re-raised, retried, and delivered to SQS after exhaustion.	Handler boundary raises InfrastructureError; destination configured; captured SQS envelope.	Implemented, verified for one injected failure
FR-15	Structured JSON logs record start and terminal states with request and object identifiers.	JsonFormatter and _log; CloudWatch log events (Figure 9.8).	Implemented and observed

2.3 Nonfunctional requirements

The nonfunctional requirements describe the qualities the system should have. Several are bounded by the V1 scope. For example, the function is designed to limit allocations before pixel decoding, but the deployed 512 MB memory boundary at the 25 MiB input ceiling has not been measured in this evidence set.

Table 2.2. *Nonfunctional requirements and current evidence.*

ID	Requirement	Design response	Evidence status
NFR-01	Private data storage	Block all public access; TLS-only bucket policy; SSE-S3 default encryption.	Implemented and documented; privacy settings not visible in the captured S3 pages.
NFR-02	Least-privilege execution	Scoped inline policy for the incoming/ prefix, sfps-processing table, sfps-failed-events queue, and function log group.	Policy documented; role exists in the console; attached policy JSON not visible in the capture.
NFR-03	Deterministic deployment	Exact runtime pins, Linux arm64 wheels, flat ZIP layout, archive verification.	Verified by direct archive inspection on 18 October 2026.
NFR-04	Idempotent processing	Deterministic processing_id and conditional writes.	Verified by automated duplicate and race tests.

ID	Requirement	Design response	Evidence status
NFR-05	Bounded input processing	25 MiB upload ceiling; 16,000,000 total image pixels; iterator-based text counts.	Implemented; local boundary tests pass; deployed 25 MiB memory and timeout boundary not measured.
NFR-06	Observable failures	Structured JSON logs, FAILED records, SQS on-failure destination.	Implemented; CloudWatch and SQS evidence captured.
NFR-07	Sanitized diagnostics	Allowlisted persisted messages; fixed boundary InfrastructureError; stack frames without source lines, locals, or exception text.	Verified by regression tests.
NFR-08	Reproducible local tests	pytest with moto; no AWS credentials or network required.	135 tests passed on 18 October 2026.
NFR-09	Code quality	Ruff lint gate on src and tests.	All checks passed on 18 October 2026.
NFR-10	Operational documentation	README, PRD, build guide, AWS console guide, ADR log, build state.	Present in the repository.
NFR-11	Cost awareness	Pay-per-use services; no idle server; log retention set to 30 days.	Design property; no billing data collected.
NFR-12	Fault isolation	Permanent failures do not retry; infrastructure failures do; a failing record stops the current batch and is retried.	Implemented and tested; mixed-batch behavior documented.

2.4 Supported formats and limits

Table 2.3. *Supported file types, validation rules, and configured limits.*

Input	Accepted extensions	Content rule	Metadata stored	Limits
JPEG image	.jpg, .jpeg (case-insensitive)	Bytes start with FF D8 FF; Pillow verifies and decodes every frame; detected format must be JPEG.	width, height, image_format	25 MiB upload ceiling; 16,000,000 total pixels across all frames.
PNG image	.png (case-insensitive)	Bytes start with 89 50 4E 47 0D 0A 1A 0A; Pillow verifies and decodes every frame; detected format must be PNG.	width, height, image_format	25 MiB upload ceiling; 16,000,000 total pixels across all frames.
UTF-8 text	.txt (case-insensitive)	Strict UTF-8 decode; one leading BOM removed; whitespace-only content is valid.	characters, words, lines	25 MiB upload ceiling; zero-byte objects rejected before decoding.
Unsupported	Any other extension, or no extension	Rejected before content download.	None; FAILED error record only	Recorded as FAILED with UNSUPPORTED_EXTENSION.

The 25 MiB ceiling is a hard cap. MAX_FILE_SIZE_BYTES may lower the limit but cannot raise it above 26,214,400 bytes. The image budget is calculated as width multiplied by height multiplied by the number of frames, and it is checked before pixel allocation. This matters for animated PNGs and for images that are small on disk but expensive to decode. Text statistics use regular-expression iterators so the function does not build per-word or per-line lists.

2.5 Error taxonomy

The error codes are part of the public contract between the function, the DynamoDB record, and an operator reading the table. Validation codes identify inputs that will not become valid on retry. The infrastructure code identifies faults that may be transient. A malformed event is kept separate because no processing identity can be derived for it, so no record can be written.

Table 2.4. *Error codes stored in the DynamoDB error map.*

Error code	Error type	Trigger	Retry behavior
ZERO_BYTE	ValidationError	HeadObject reports 0 bytes.	No retry; FAILED record.
SIZE_EXCEEDED	ValidationError	Object is larger than MAX_FILE_SIZE_BYTES.	No retry; FAILED record.
UNSUPPORTED_EXTENSION	ValidationError	Extension is not jpg, jpeg, png, or txt.	No retry; FAILED record.
TYPE_MISMATCH	ValidationError	Content signature or detected format contradicts the extension.	No retry; FAILED record.
CORRUPT_IMAGE	ValidationError	Pillow verification or full decode fails for a matching signature.	No retry; FAILED record.
IMAGE_DIMENSIONS_EXCEEDED	ValidationError	Total frame pixels exceed 16,000,000 or Pillow decompression-bomb protection fires.	No retry; FAILED record.
INVALID_ENCODING	ValidationError	Text content is not valid strict UTF-8.	No retry; FAILED record.
PREFIX_NOT_ALLOWED	ValidationError	Object key is outside incoming/.	No retry; FAILED record.
MALFORMED_EVENT	MalformedEventError	Event or record cannot be safely interpreted, or a required identity field is missing or the wrong type.	Raise immediately; no record for that identity; async retries and SQS may follow.
INFRASTRUCTURE	InfrastructureError	AWS service error, unexpected exception, or invalid configuration.	Log, attempt guarded FAILED update, re-raise; async retries and SQS follow.

2.6 Security expectations

The bucket is private. Public access is blocked at the bucket level, a bucket policy denies requests that do not use TLS, and default encryption uses SSE-S3. The function reads only objects under the incoming/ prefix and uses GetObject and GetObjectVersion, which also authorize HeadObject. The execution role can read and write only the processing table, send only to the failure queue, and write only to the function's log group. The role cannot create log groups, so the log group is created before deployment.

Uploaded content is treated as hostile. The function never executes a file or trusts its extension. It checks the signature and decodes images with Pillow before accepting metadata. It decodes text strictly. Error messages persisted to DynamoDB come from a fixed allowlist. Unexpected exceptions are logged as sanitized stack frames without source lines, local variables, or raw exception text. The Lambda boundary raises a fixed-message InfrastructureError so SDK or decoder messages do not reach the runtime log or the destination responsePayload.

2.7 Operational constraints

The deployment is manual in V1. Every resource is created in one AWS region, and the region must be used consistently in ARNs, policies, and the queue destination. The deployment archive must be built on Windows with Linux arm64 wheels and uploaded as a ZIP file under the 50 MB direct-upload limit. The function has a 30-second timeout and 512 MB of memory. The log group must exist before the first invocation because the execution role deliberately lacks logs:CreateLogGroup. S3 notification changes can take several minutes to become active, and bucket versioning has an initial propagation window before the first object operations.

These constraints are not defects. They are consequences of the chosen V1 scope and the least-privilege design. The deployment guide records them so that a future operator does not mistake a propagation delay or a missing log group for an application bug.

2.8 Requirements traceability and evidence status

The PRD describes intended behavior. It is not, by itself, evidence that a requirement was implemented or tested. This case study uses three checks. The source code and tests establish what the repository implements. The console captures establish which resources exist and how they are configured. The deployed test fixtures and the SQS envelope establish what the deployed system did for a specific set of inputs. A requirement is marked verified only when the relevant check supports it.

The BUILD_STATE document dated 16 October 2026 recorded deployment acceptance as pending. The console captures and the SQS envelope are dated 18 October 2026, so they post-date that snapshot and confirm that a deployment and a test run occurred. They do not prove every item in the acceptance guide. In particular, the 25 MiB memory and timeout boundary, the exact attached IAM policy, the 30-day log retention value, and the per-file error codes in the truncated DynamoDB console view remain unverified or only partially verified. The evidence inventory records those gaps explicitly.

Chapter 3. Technology Stack

3.1 Selection principles

The stack follows four principles. Use managed services for the undifferentiated parts of the pipeline: storage, compute invocation, state, queueing, logging, and identity. Keep the application code small enough to review in one sitting. Pin every runtime dependency so the deployed environment is deterministic. Choose tools that work on the available Windows development host without hiding the Linux target.

The result is a deliberately conventional AWS stack. There is no framework between the S3 event and the Python handler, no ORM over DynamoDB, and no container image. The function uses boto3 directly and Pillow for image validation. This makes the behavior visible in the source and keeps the deployment package understandable.

3.2 AWS services

Amazon S3 stores the uploaded objects and emits the event that starts processing. Versioning is enabled because the processing identity depends on the version ID. The bucket is private, encrypted with SSE-S3, and protected by a TLS-only bucket policy. A lifecycle rule expires noncurrent versions after 30 days. S3 never receives a write from the function; the function only reads the pinned object version.

AWS Lambda runs the processing function. A single function handles all supported types and dispatches by validated extension. The runtime is Python 3.13 on arm64, with 512 MB of memory and a 30-second timeout. The function is invoked asynchronously by S3, which is why Lambda's retry count and on-failure destination apply. The handler is handler.lambda_handler, and the deployment package places handler.py at the archive root.

Amazon DynamoDB stores one processing record per S3 object version. The table uses on-demand capacity, a partition key named processing_id with type String, no sort key, no secondary indexes, and no point-in-time recovery in V1. Conditional writes provide the idempotency guarantee. The table is the operational ledger: an operator can see whether a version is PROCESSING, COMPLETED, or FAILED, and can read the metadata, checksum, or error map.

Amazon SQS receives asynchronous invocation failures that exhaust Lambda's function-error retries. The queue is a standard queue named sfps-failed-events, with 14-day message retention, a 30-second visibility timeout, SSE-SQS encryption, and no dead-letter queue. No consumer runs in V1. The queue exists so that a failed event is not lost when the function cannot complete it.

Amazon CloudWatch Logs stores the function's structured JSON application logs and the Lambda platform START, END, and REPORT lines. The log group is /aws/lambda/sfps-processor and is pre-created with 30-day retention. Application logs use one JSON object per line and include request_id, event_id, processing_id, bucket, object_key, version_id, file_type, file_size, status, processing_duration, and error fields where applicable.

AWS IAM defines the execution boundary. The role trusts lambda.amazonaws.com and carries a scoped inline policy. The policy allows s3:GetObject and s3:GetObjectVersion on the bucket's incoming/ prefix, dynamodb:GetItem, PutItem, and UpdateItem on the processing table, sqs:SendMessage on the failure queue, and logs:CreateLogStream and logs:PutLogEvents on the function's log group. There are no broad administrator permissions and no wildcard resources beyond the prefix and log-group suffixes required by the design.

3.3 Python and libraries

Python 3.13 is used for local development, the Ruff target, and the Lambda runtime. The project standardized on one interpreter version to avoid a local-versus-deployed mismatch. The runtime dependency set is intentionally small: boto3 and botocore for AWS calls, jmespath and s3transfer as boto3 dependencies, Pillow for image validation and metadata, python-dateutil and six as transitive dependencies, and urllib3 as the HTTP layer. The standard library supplies hashlib, json, logging, os, time, datetime, io, re, warnings, and urllib.parse.

Pillow performs three jobs in the image path. It verifies the image structure, it reopens the same in-memory bytes and decodes every frame, and it reports the detected format and dimensions. The validation layer checks the signature before Pillow is called, checks the pixel budget before allocation, and maps expected decoder errors to CORRUPT_IMAGE. Unexpected errors such as MemoryError propagate to the infrastructure path instead of being mislabeled as a bad file.

Development tools are pytest for tests, moto for mocked S3 and DynamoDB, and Ruff for linting. The tests run offline with no AWS credentials or network access. The handler is reloaded inside a moto context so the boto3 clients created at import time are intercepted. The test fixtures build synthetic images and versioned S3 objects, which keeps the suite fast and deterministic.

3.4 Development and deployment tools

Development runs on Windows 11 with PowerShell 5.1 and a Python 3.13 virtual environment. Dependencies are installed with pip from exact pins. The deployment package is assembled with pip's cross-platform wheel download, a flat copy of src/, and Python's zipfile module. The zip is verified for forward-slash entry names, a root-level handler.py, the presence of aarch64 extension modules, the absence of bytecode, and the 50 MB direct-upload limit. The AWS CLI uploads the archive and supports selected verification commands. The AWS Management Console is used for resource creation, inspection, and the acceptance walkthrough.

PowerShell 5.1 has two practical traps in this workflow. It does not support && or || command chaining, so the deployment commands use semicolons and explicit exit-code checks. Its Compress-Archive cmdlet writes backslash separators into ZIP entry names, which Lambda cannot unpack correctly. The project uses Python's zipfile with an explicit separator replacement and a verification assert instead.

3.5 Technology interactions

Table 3.1. *Technology responsibilities and interactions.*

Technology	Responsibility	Why it was selected	Interacts with
Amazon S3	Private source-file storage, versioning, event notification.	Durable object storage with native event integration and version IDs.	Lambda trigger, IAM read policy, lifecycle rule.
AWS Lambda	Validation, metadata extraction, checksum, state transitions.	Per-event compute without a running server; native async retries and destinations.	S3 event, DynamoDB writes, CloudWatch logs, SQS destination.
Amazon DynamoDB	One processing record per object version.	Serverless key-value store with conditional writes and on-demand capacity.	Lambda read/write calls, IAM table policy.
Amazon SQS	Capture exhausted asynchronous invocation failures.	Durable buffer for events that cannot be processed after retries.	Lambda on-failure destination, IAM send policy.
Amazon CloudWatch Logs	Structured application logs and platform diagnostics.	Native Lambda integration, retention control, searchable log streams.	Lambda execution role, operators.
AWS IAM	Execution identity and least-privilege boundary.	Scoped permissions for each AWS call the function makes.	Lambda execution role, S3, DynamoDB, SQS, CloudWatch.
Python 3.13	Application runtime.	Matches the Lambda runtime and the local development interpreter.	boto3, Pillow, standard library.
boto3 / botocore	AWS SDK calls.	Official SDK with explicit request and error semantics.	S3, DynamoDB, CloudWatch.
Pillow	Image verification, decoding, metadata.	Mature decoder with format detection and decompression-bomb protection.	Validation and image processor.
pytest / moto / Ruff	Automated tests and lint.	Fast offline feedback on AWS orchestration and pure logic.	Local development and CI-style checks.
PowerShell / pip / zipfile	Build and package the Lambda archive.	Available on the Windows host; produces a Linux-compatible ZIP.	AWS CLI upload.

3.6 Alternatives considered

A long-running worker on EC2 or ECS was rejected for V1 because it would add capacity management and idle cost without improving the core demonstration. API Gateway and a web frontend were rejected because S3 already provides an event source and the project is about processing, not upload UX. AWS SAM, CloudFormation, and Terraform were deferred because the PRD explicitly requires a manual deployment that the owner can follow in the console. A KMS customer-managed key was deferred because SSE-S3 meets the stated encryption requirement without adding key-policy complexity. PDF and video processing were excluded because the validation and metadata rules would expand the scope substantially.

The architecture decision log records seventeen decisions, including the choice of exact pins, the manylinux_2_28_aarch64 platform tag, the failure classification, the strong duplicate read, and the decision to omit logs:CreateLogGroup. The most consequential decisions are summarized in Chapter 4 and revisited in Chapter 11.

Chapter 4. System Architecture

4.1 Architectural overview

SFPS is a linear event-driven pipeline with one fan-out for diagnostics and one failure branch. An uploader writes an object under the incoming/ prefix of a private, versioned S3 bucket. S3 emits an ObjectCreated event with the bucket name, object key, size, version ID, and sequencer. The event asynchronously invokes the sfps-processor Lambda function. The function claims a processing record in DynamoDB, reads the object size with a version-pinned HeadObject, validates the extension and size, fetches the object bytes once with a version-pinned GetObject, validates the content, extracts metadata, computes SHA-256, and writes the terminal state. Application logs go to CloudWatch. If an infrastructure failure exhausts the asynchronous retry budget, Lambda sends an invocation envelope to the sfps-failed-events SQS queue.

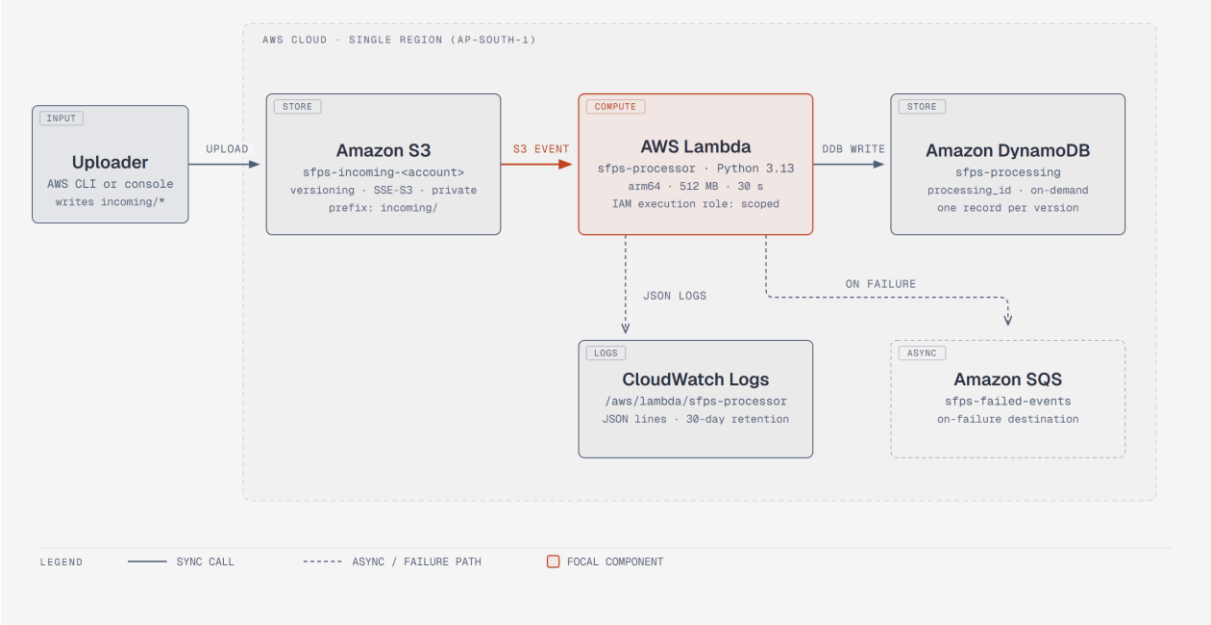


Figure 4.1. *Original SFPS event-driven architecture diagram, drawn from the implemented code and deployed configuration.*

The diagram is an original explanatory figure, not an AWS Console screenshot. The console capture in Figure 4.3 shows the same primary integration from the deployed function's point of view: S3 on the left as the trigger, the sfps-processor function in the center, and SQS on the right as the on-failure destination.

The pipeline is intentionally sequential within one invocation. A single S3 event can carry multiple records, and the handler processes them in order. A permanent validation failure is recorded and the loop continues. An infrastructure failure stops the batch and is re-raised. On retry, records that already reached COMPLETED are skipped, and the failing record is attempted again. Records later in the batch are not claimed until the failing record succeeds or becomes a permanent validation failure.

4.2 The event-driven workflow

A single upload follows this sequence:

1. The uploader writes the object to s3://sfps-incoming-<account-id>/incoming/... using the console or the AWS CLI.
2. S3 stores the object as a new version and emits an ObjectCreated:* notification. The notification includes the bucket name, URL-encoded object key, object size, version ID, and sequencer.
3. Lambda receives an asynchronous invocation. The handler parses the Records array and validates the bucket, key, and version ID types before deriving any identity.
4. The handler computes processing_id from the canonical JSON encoding of bucket, object key, and version ID, then attempts a conditional PutItem with status PROCESSING.
5. If the claim succeeds, the handler checks the incoming/ prefix, reads the authoritative size with HeadObject using the event version ID, rejects zero-byte and oversized objects, and validates the extension allowlist.
6. For a supported type, the handler fetches the object once with GetObject using the same version ID, validates the content, extracts metadata, and computes SHA-256 over the bytes.
7. The handler performs a conditional UpdateItem to COMPLETED, guarded so an existing COMPLETED record cannot be overwritten.
8. The handler writes one JSON start line and one JSON terminal line to CloudWatch. A validation failure writes a FAILED record and returns successfully. An infrastructure failure logs sanitized diagnostics, attempts a guarded FAILED update, and re-raises.

4.3 Resource topology and trust boundaries

The trust boundary is the AWS account and the region. The bucket, function, table, queue, and log group all live in ap-south-1. The function assumes an execution role through the Lambda service principal. The role's permissions are limited to the incoming/ prefix of the project bucket, the sfps-processing table, the sfps-failed-events queue, and the function's log group. The uploader is outside the processing boundary: it needs permission to write to the bucket, but the function never needs permission to write back to S3.

There are two data paths. The processing path carries object bytes from S3 to Lambda memory, then derived metadata and a checksum to DynamoDB. The diagnostic path carries log events from Lambda to CloudWatch. The failure path carries an invocation envelope from the Lambda asynchronous invocation system to SQS. The function does not send to SQS directly; the destination is configured on the function and is delivered by Lambda after retries are exhausted. This distinction matters when reading the envelope: requestPayload contains the entire original event, while requestContext describes the invocation, condition, and approximate invoke count.

4.4 Processing sequence

Figure 4.2 shows the sequence for one object version. The first message is the asynchronous S3 event. The Lambda activation bar spans the rest of the processing. The function claims the DynamoDB record before reading S3, which means a claim failure can stop processing before any object bytes are fetched. The size check is a HeadObject, not a download. The content read is a single GetObject. Validation, decoding, and hashing happen in memory on that one byte sequence. The terminal write is the only state transition after the claim.

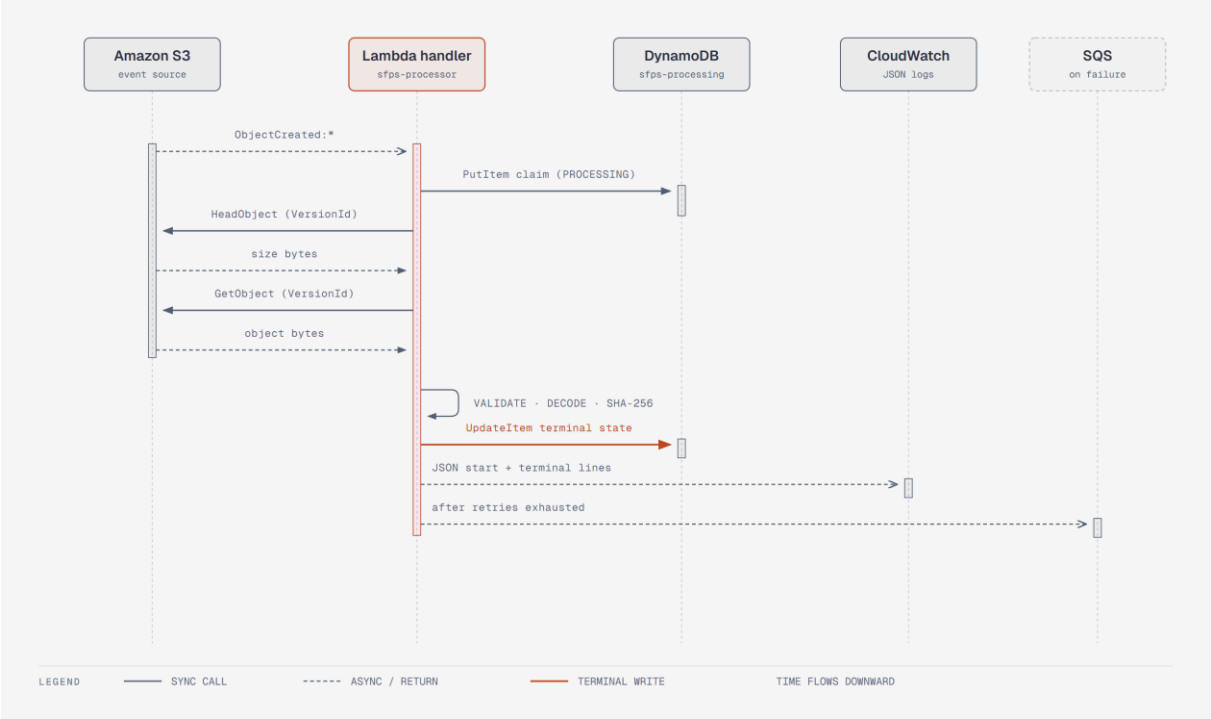


Figure 4.2. *Processing sequence for one S3 object version, from event delivery through the terminal DynamoDB write.*

The diagram omits the internal branch where a duplicate COMPLETED record is skipped before any S3 call. That path returns from the claim step, logs a duplicate message, and performs no HeadObject or GetObject. It is described in Chapter 5 with the conditional write protocol.

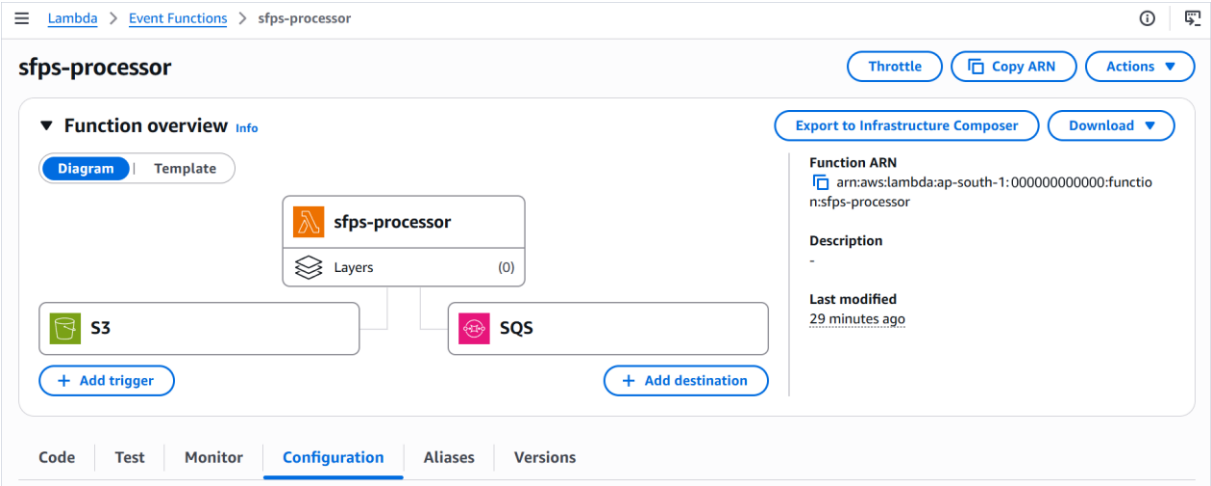


Figure 4.3. *Deployed Lambda function overview showing the S3 trigger on the left and the SQS destination on the right.*

4.5 Object versioning and version-pinned reads

Versioning is not a background convenience in SFPS. It is part of the processing identity. When an object is overwritten, S3 creates a new version ID. The event for the new upload carries the new version ID, so the function treats it as a separate processing identity and writes a separate record. The old version remains addressable until the lifecycle rule expires it. This is why the DynamoDB scan in Chapter 9 contains two sample.png records with different sizes and outcomes: they are two versions of the same key, not a duplicate write.

Every read uses the version ID from the event. HeadObject is called with VersionId, so the authoritative size belongs to the exact version that triggered the event. GetObject is called with the same VersionId, so the bytes that are validated, hashed, and measured are the

bytes identified by the record. There is no fallback to the latest version. If the version ID is missing, empty, or the string null, the record is malformed and the handler raises before writing an identity. A latest-version fallback would make the record describe a different object than the event, which is worse than failing loudly.

The lifecycle rule permanently deletes noncurrent versions after 30 days. This bounds storage growth from repeated uploads while keeping recent versions available for investigation. It does not affect current versions and it does not change the processing identity of a new upload.

4.6 Nested prefixes

The trigger prefix is incoming/, not a flat folder. S3 keys are strings, so incoming/2026/photo.jpg and incoming/users/demo/file.txt are valid keys under the same prefix. The handler checks that the decoded key starts with incoming/, extracts the file name from the last path segment, and derives the extension from that segment. Dots in directory names are irrelevant. The positive test suite includes a nested/ folder and a completed 06-nested-image.jpg record, which shows that the deployed path handling works beyond a single-level upload.

4.7 Failure classification

The failure model is the most important design decision in the project. A validation failure means the input is permanently unacceptable. The function records FAILED with a fixed error code, logs the outcome, returns successfully, and continues with the next record. Retrying the same bytes would produce the same result, so a retry would only waste invocations and delay the queue. An infrastructure failure means the function could not complete the work because of an AWS service error, an unexpected exception, or a configuration problem. The function logs sanitized diagnostics, attempts a guarded FAILED update when a record exists, and re-raises. Lambda then applies its asynchronous retry policy, and the event can reach SQS.

Table 4.1. *Failure classes and their behavior.*

Class	Examples	DynamoDB result	Lambda result	Retry and destination
Permanent validation	Unsupported extension, type mismatch, corrupt image, invalid UTF-8, zero bytes, oversized object, wrong prefix.	FAILED with allowlisted error map.	Returns success; batch continues.	No retry from this record. A permanent-only event does not reach SQS.
Transient or infrastructure	S3 error, DynamoDB error, unexpected decoder error, configuration error.	Guarded FAILED update if the record exists; otherwise no record.	Raises a fixed-message InfrastructureError.	Async retries, then the original event can reach the SQS on-failure destination.
Malformed event	Missing Records, missing bucket or key, missing or invalid version ID, wrong field type.	No record, because identity is undefined.	Raises MalformedEventError immediately.	Async retries, then the original event can reach the SQS destination.

A mixed batch complicates the story. If one record fails permanently and a later record fails with an infrastructure error, the entire original event is retried and can be delivered to SQS. The destination envelope then contains every original record, including the permanent failure that did not itself request a retry. This is a property of Lambda's asynchronous invocation model, not a bug in the classification. The PRD and the README both call it out.

4.8 Asynchronous retries and the failure destination

S3 invokes Lambda asynchronously. The function's asynchronous invocation configuration sets MaximumRetryAttempts to 2, which allows the initial attempt plus up to two function-error retries. Throttling and Lambda system errors use separate retry behavior up to the maximum event age, which is left at the default of six hours. Duplicate delivery remains possible. The on-failure destination is configured as an SQS destination with condition On failure, not as a classic dead-letter queue.

The captured SQS message in Chapter 9 confirms the envelope shape for one injected failure. requestContext.condition is RetriesExhausted, approximateInvokeCount is 3, requestPayload contains the original S3 event for incoming/images.jpg, and responsePayload contains a sanitized InfrastructureError with the fixed message and a stack trace that stops at the handler boundary. That is the expected destination behavior. It does not prove that every possible failure mode produces the same envelope, and it does not prove that the retry timing matches a particular schedule.

4.9 Observability

CloudWatch receives two kinds of log lines. The Lambda platform writes START, END, and REPORT lines for each invocation. The application writes one JSON object per line through the sfps logger. A successful record produces a PROCESSING start line and a COMPLETED terminal line. A permanent failure produces a PROCESSING line and a FAILED terminal line. A duplicate skip produces an explicit duplicate message. An infrastructure failure produces a diagnostic line with sanitized stack frames and then the boundary exception. The application logger sets propagate to False, so the same record is not duplicated through the runtime root logger.

The log group is pre-created with 30-day retention. The execution role can create log streams and put log events but cannot create the log group, which is why the group must exist before the first invocation. The captured log group page shows the group and its Standard log class; the retention value is not visible in that capture, so the 30-day setting is documented rather than visually verified.

4.10 Architecture status and evidence

The architecture is implemented and deployed. The console captures show the S3 trigger, the Lambda function, the DynamoDB table, the CloudWatch log group, the IAM role page, and the SQS destination. The SQS envelope shows a real exhausted failure. The parts that remain unverified are configuration details that the captures do not expose, such as the exact attached IAM policy, the bucket policy text, the lifecycle rule, and the log retention value. Those are documented in the repository and the deployment guide, and they are listed in the evidence inventory rather than presented as visual proof.

Chapter 5. Application Design and Processing Logic

5.1 Source organization

The repository separates AWS orchestration from pure file logic. handler.py owns event parsing, AWS calls, the conditional write protocol, error categorization, configuration, and logging. validation.py owns extension, signature, image, and text validation. checksum.py owns the SHA-256 helper. processors/image.py and processors/text.py own metadata extraction. The pure modules never import boto3, so they can be tested without mocks. The handler tests use moto to intercept S3 and DynamoDB and invoke the handler with synthetic S3 events.

Code 5.1. *Repository layout for the Lambda source and tests.*

```
src/
  handler.py          AWS orchestration, config, logging, write protocol
  validation.py       extension, signature, image, and UTF-8 validation
  checksum.py         SHA-256 helper
  processors/
    image.py          image dimensions and format
    text.py           character, word, and line counts
tests/
  test_handler.py     moto-mocked handler and protocol tests
  test_validation.py  pure validation tests
  test_image.py       image metadata tests
  test_text.py        text metadata tests
  test_checksum.py    known-digest tests
  test_audit_regressions.py  audit and resource-bound regression tests
```

5.2 Configuration contract

The function reads three environment variables at import time. TABLE_NAME is required and must be non-empty. MAX_FILE_SIZE_BYTES is optional and defaults to 26,214,400 bytes; it must parse as an integer from 1 through 26,214,400. LOG_LEVEL is optional and defaults to INFO; it must be a valid Python logging level name. Invalid configuration raises ConfigurationError at import, which is intentionally loud. The error message does not echo the raw value or a chained parsing exception.

Table 5.1. *Lambda environment variables.*

Variable	Required	Default	Validation	Failure behavior
TABLE_NAME	Yes	None	Non-empty after trimming whitespace.	ConfigurationError at import; async retries; SQS destination after exhaustion.
MAX_FILE_SIZE_BYTES	No	26214400	Integer from 1 through 26214400 inclusive.	ConfigurationError at import; no raw value in the message.
LOG_LEVEL	No	INFO	Valid Python logging level name, case-insensitive.	ConfigurationError at import; no raw value in the message.

5.3 Handler lifecycle

The module loads configuration and creates one boto3 S3 client and one DynamoDB Table handle at import time. Lambda reuses the execution environment across invocations, so the clients are created once per cold start and reused. The logger is configured once, with a JSON formatter, propagate set to False, and handlers cleared before a new one is attached so module reloads remain idempotent under pytest. The handler entry point parses the Records array, loops over the records sequentially, and wraps the loop in a boundary that converts unexpected exceptions into a fixed-message InfrastructureError.

5.4 Event parsing and processing identity

The handler accepts only an event with a non-empty Records array. Each record must be an object with an s3 section, a bucket name, an object key, and a version ID. Bucket, key, and version must be non-empty strings; the version cannot be whitespace-only or the literal string null. The optional sequencer must be a string when present. The key is decoded with unquote_plus, so %XX escapes and plus signs decode correctly. A malformed record raises MalformedEventError before any identity is derived or any AWS write is attempted.

The processing identity is a SHA-256 digest of canonical JSON containing the bucket, object key, and version ID. Canonical JSON uses sorted keys and compact separators, which makes the encoding injective for arbitrary key strings. Naive concatenation would be ambiguous if keys contained the delimiter. The result is a fixed 64-character hexadecimal string used as the DynamoDB partition key.

Code 5.2. *Deterministic processing identity from handler.py.*

```
def _processing_id(bucket: str, object_key: str, version_id: str) -> str:
    identity = json.dumps(
        {"bucket": bucket, "object_key": object_key, "version_id": version_id},
        sort_keys=True,
        separators=(",", ":"),
    )
```

```
    return hashlib.sha256(identity.encode("utf-8")).hexdigest()
```

The file name is the last path segment of the decoded key. The file type is the lowercased extension without the dot, or absent when the file name has no extension. The extension parser uses rpartition on the last segment, so dots in directory names do not confuse it. A trailing dot with no characters after it returns no extension.

5.5 Validation workflow

Validation is ordered to avoid unnecessary work and to make the error code unambiguous. The prefix check runs first because a key outside incoming/ cannot be a legitimate event for this pipeline. The size check runs before the extension check so a zero-byte folder marker is reported as ZERO_BYTE rather than UNSUPPORTED_EXTENSION. The extension check runs before any content download, so an unsupported PDF is rejected without reading its 1.6 MB body. Content validation runs after the single GetObject.

Table 5.2. *Validation order and failure mapping.*

Step	Check	Input	Failure code	AWS call
1	Record structure and identity types	Records array, s3 object, bucket, key, version, sequencer.	MALFORMED_EVENT	None.
2	Prefix	Decoded object key starts with incoming/.	PREFIX_NOT_ALLOWED	None; claim may already exist.
3	Authoritative size	HeadObject ContentLength for the pinned version.	ZERO_BYTE or SIZE_EXCEEDED	HeadObject with VersionId.
4	Extension allowlist	Lowercased extension in jpg, jpeg, png, txt.	UNSUPPORTED_EXTENSION	None; no download.
5	Content fetch	One GetObject for the pinned version.	INFRASTRUCTURE on service error	GetObject with VersionId.
6	Type-specific validation	JPEG or PNG signature, pixel budget, verify, decode; or strict UTF-8.	TYPE_MISMATCH, CORRUPT_IMAGE, IMAGE_DIMENSIONS_EXCEEDED, INVALID_ENCODING	None; in-memory.
7	Metadata and checksum	Dimensions/format or character/word/line counts; SHA-256 of bytes.	INFRASTRUCTURE on unexpected error	None; in-memory.
8	Terminal write	Conditional UpdateItem to COMPLETED or FAILED.	INFRASTRUCTURE on service error	UpdateItem.

5.6 Image validation and metadata

Image validation does not rely on Pillow alone. The handler first checks the extension against the allowlist, then the bytes against the claimed signature: FF D8 FF for JPEG and 89 50 4E 47 0D 0A 1A 0A for PNG. A mismatch is TYPE_MISMATCH before any decoder work. If the signature matches, the validator opens the image, checks the total pixel budget, calls verify, closes the image, reopens the same in-memory bytes, checks the budget again, and decodes every frame. The detected format must match the extension. Expected malformed-image errors become CORRUPT_IMAGE. Decompression-bomb warnings and errors become IMAGE_DIMENSIONS_EXCEEDED. Unexpected errors such as MemoryError propagate to the infrastructure path.

Code 5.3. *Image validation order from validation.py.*

```
expected_format = _IMAGE_FORMAT_BY_EXTENSION[file_type]
signature = JPEG_SIGNATURE if expected_format == "JPEG" else PNG_SIGNATURE
if not file_bytes.startswith(signature):
    raise ValidationError("TYPE_MISMATCH", ...)
try:
    with warnings.catch_warnings():
        warnings.simplefilter("error", Image.DecompressionBombWarning)
        with Image.open(io.BytesIO(file_bytes)) as image:
            check_image_budget(image)
            detected_format = image.format
            image.verify()
        with Image.open(io.BytesIO(file_bytes)) as image:
            check_image_budget(image)
            for frame in range(getattr(image, "n_frames", 1)):
                image.seek(frame)
                image.load()
except (Image.DecompressionBombError, Image.DecompressionBombWarning) as exc:
    raise ValidationError("IMAGE_DIMENSIONS_EXCEEDED", ...) from exc
except (UnidentifiedImageError, OSError, SyntaxError, EOFError, ValueError) as exc:
    raise ValidationError("CORRUPT_IMAGE", ...) from exc
```

The metadata processor reopens the validated bytes to read width, height, and image.format. It checks the budget again and returns a dictionary with exactly those three keys. The reopen is an in-memory decode, not another S3 download. The single-read requirement refers to one version-pinned GetObject, not to one Pillow open.

5.7 Text validation and metadata

Text validation decodes the bytes as strict UTF-8. Invalid sequences become INVALID_ENCODING. A single leading byte-order mark (U+FEFF) is removed after decoding. A BOM-only payload becomes an empty string at this layer; a truly zero-byte object is rejected earlier by the size check. The text processor counts Unicode code points for characters, whitespace-separated runs for words, and

splitlines-style boundaries for lines. A trailing newline does not create an extra line. Whitespace-only text is valid and may contain zero words.

The counters use regular-expression iterators rather than split or splitlines lists. A 200,000-line stress test in the regression suite confirms that the extra allocation stays below 1 MiB. This is a deliberate resource-bound decision: a 25 MiB text file with millions of short words should not create a list proportional to the word count.

5.8 SHA-256

The checksum is computed over the original bytes returned by the version-pinned GetObject. It is not computed over re-encoded image data, decoded text, or metadata. This makes the stored digest a fingerprint of the uploaded object version. The checksum is metadata in V1; the system does not use it for content-level deduplication. A known-digest test anchors the helper to the standard SHA-256 values for empty bytes and for the string abc.

5.9 DynamoDB record schema

Every processing record has the same identity fields and one of three statuses. The terminal transition adds file_size and processed_at. A COMPLETED record adds sha256, metadata, and file_type, and removes any stale error. A FAILED record adds an error map and, when known, file_size and file_type. The error map has exactly error_code, error_type, and message. Its serialized size stays within 1,024 characters; only the message is truncated if necessary.

Table 5.3. *DynamoDB record attributes.*

Attribute	Type	Present	Description
processing_id	String	Always	SHA-256 hex of the canonical identity encoding; 64 characters.
bucket	String	Always	Source bucket name.
object_key	String	Always	Decoded object key.
version_id	String	Always	S3 version ID of the processed object.
file_name	String	Always	Last path segment of the object key.
file_type	String	When known	Lowercased extension without the dot.
file_size	Number	Terminal	Size in bytes from HeadObject.
sha256	String	COMPLETED	SHA-256 of the original version-pinned bytes.
status	String	Always	PROCESSING, COMPLETED, or FAILED.
created_at	String	Always	UTC ISO-8601 timestamp ending in Z; set at claim time and never overwritten.
processed_at	String	Terminal	UTC ISO-8601 timestamp ending in Z; set at the terminal transition.
metadata	Map	COMPLETED	width, height, image_format for images; characters, words, lines for text.
error	Map	FAILED	error_code, error_type, message.

5.10 Processing states

A record begins in PROCESSING when the claim PutItem succeeds. It ends in COMPLETED after a successful validation, metadata extraction, checksum, and terminal update, or in FAILED after a permanent validation failure or an infrastructure failure that has a record to update. A PROCESSING record can be re-attempted by a later delivery. A FAILED record can also be re-attempted. A COMPLETED record is never re-attempted. The terminal update is guarded, so a late duplicate cannot overwrite a completed result.

Table 5.4. *Processing states and transitions.*

State	Entered by	Allowed next state	Duplicate behavior
PROCESSING	Conditional PutItem at claim.	COMPLETED or FAILED.	Re-attempted without overwriting created_at.
COMPLETED	Conditional terminal UpdateItem after successful processing.	None.	Skipped and logged; no S3 fetch.
FAILED	Conditional terminal UpdateItem after a validation or infrastructure failure.	COMPLETED on a later successful re-attempt.	Re-attempted.

5.11 Conditional writes and idempotency

The write protocol has two conditional operations. The claim is a PutItem conditioned on attribute_not_exists(processing_id). If the condition fails, the handler reads the existing item with ConsistentRead=True. If the existing status is COMPLETED, the handler logs a duplicate and returns without touching S3. If the status is PROCESSING or FAILED, the handler continues the re-attempt without overwriting created_at. The terminal operation is an UpdateItem conditioned on attribute_exists(processing_id) AND status <> COMPLETED. A terminal conditional failure means another invocation already completed the record; it is logged and treated as already satisfied.

Code 5.4. *Claim protocol and duplicate handling from handler.py.*

```
try:
    _processing_table.put_item(
        Item=item,
        ConditionExpression="attribute_not_exists(processing_id)",
    )
    return True
except ClientError as exc:
    if not _is_conditional_check_failed(exc):
```

```
        raise
existing = _processing_table.get_item(
    Key={"processing_id": processing_id}, ConsistentRead=True
).get("Item")
return not (existing is not None and existing.get("status") == "COMPLETED")
```

The consistent read matters. An eventually consistent read could return a stale PROCESSING item for a version that another invocation has just completed, and the handler would reprocess it. ConsistentRead closes that window. The regression suite includes a simulated stale replica that would have triggered an S3 fetch; the handler must not fetch S3 in that case.

5.12 Failure classification and sanitized logging

The handler catches ValidationError separately from every other exception. A validation error writes FAILED through the guarded terminal update, logs a terminal FAILED line, and returns. If the FAILED write itself fails, the handler logs a secondary failure and re-raises so the batch can retry. Any other exception is logged with a diagnostic line that includes sanitized stack frames, then a guarded FAILED write is attempted when the record exists, and the exception is re-raised. At the outer Lambda boundary, the handler raises a fixed-message InfrastructureError from None, so raw SDK or decoder text does not escape through the runtime log or the destination responsePayload.

Code 5.5. *Error entry construction and the fixed infrastructure classification.*

```
if isinstance(error, validation.ValidationError) and error.error_code in (
    validation.VALIDATION_MESSAGES
):
    entry = {
        "error_code": error.error_code,
        "error_type": "ValidationError",
        "message": validation.VALIDATION_MESSAGES[error.error_code],
    }
else:
    entry = {
        "error_code": InfrastructureError.error_code,
        "error_type": InfrastructureError.error_type,
        "message": "Infrastructure or unexpected processing failure; see diagnostic logs",
    }
```

The JSON formatter keeps diagnostic frame filenames, function names, and line numbers, and omits exception text, chained exception text, source lines, and local variable values. The persisted message is chosen from an allowlist before the 1,024-character cap is applied. The cap truncates only the message field; error_code and error_type are never truncated. The application logger does not propagate to the runtime root logger, which prevents a second unstructured copy of each record.

5.13 Design limits and deliberate omissions

The handler does not write to S3, move objects, delete objects, or resize images. It does not scan for malware. It does not deduplicate content by SHA-256. It does not use a DynamoDB transaction across multiple records. It does not implement a retry loop inside the function; retries are delegated to Lambda's asynchronous invocation behavior. Each omission is a scope decision, not an accident. The V1 design favors a small, auditable processing path over a larger feature set that would be harder to verify.

Chapter 6. AWS Infrastructure Implementation

6.1 Deployment model

V1 is deployed manually in a single region. The project owner creates each resource in the AWS Management Console, using AWS CLI commands for package upload and selected verification steps. The deployment guide in the repository lists every console path, policy document, and acceptance step. The captures in this chapter show the resources that were created and the values that are visible in the console.

The region used in the captured deployment is ap-south-1 (Asia Pacific, Mumbai). Every ARN, queue URL, and policy resource in the deployment must use that region. The account identifier is written as <account-id> in this document. The bucket name is sfps-incoming-<account-id> in the documentation; the captures show the deployed name with the account number.

6.2 Amazon S3 bucket and configuration

The bucket is a general-purpose bucket with Block all public access enabled, versioning enabled, and default encryption set to SSE-S3. A bucket policy denies every request whose aws:SecureTransport condition is false, which enforces TLS. A lifecycle rule named expire-noncurrent-30d permanently deletes noncurrent versions after 30 days. The S3 event notification is configured on the Lambda side with the incoming/ prefix and all object-created events.

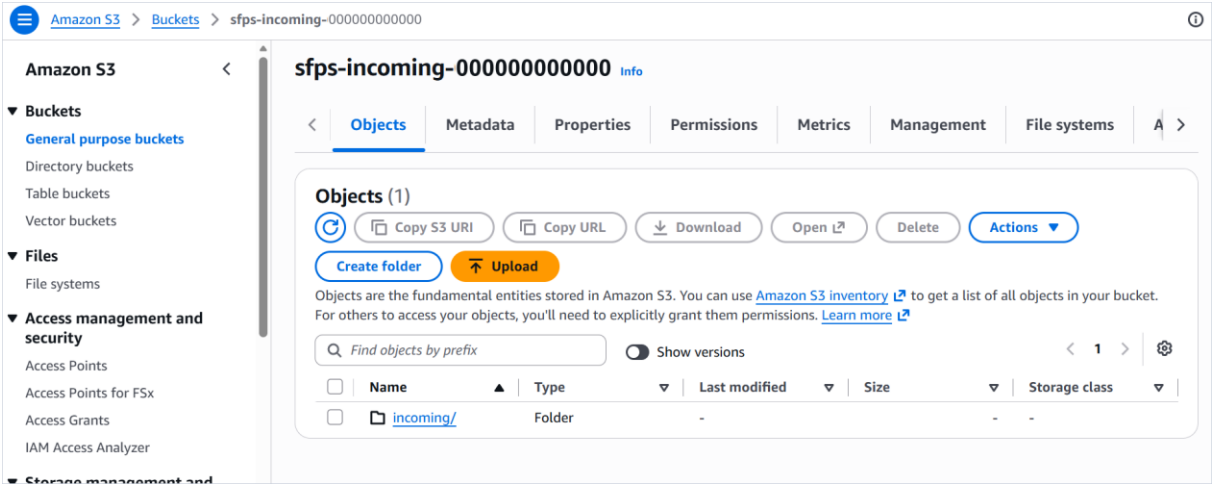


Figure 6.1. *S3 bucket object view showing the incoming/ prefix at the bucket root.*

Figure 6.1 establishes the bucket name and the top-level prefix. Figure 6.2 opens the prefix and shows the acceptance files. The sizes and names are consistent with the DynamoDB records later in the document: sample.jpg is 137.8 KB, sample.png is part of the image set, sample.pdf is 1.6 MB, and images.jpg is 44.1 KB. The partially visible row at the top of the folder is the nested sample.txt upload described in the acceptance walkthrough.

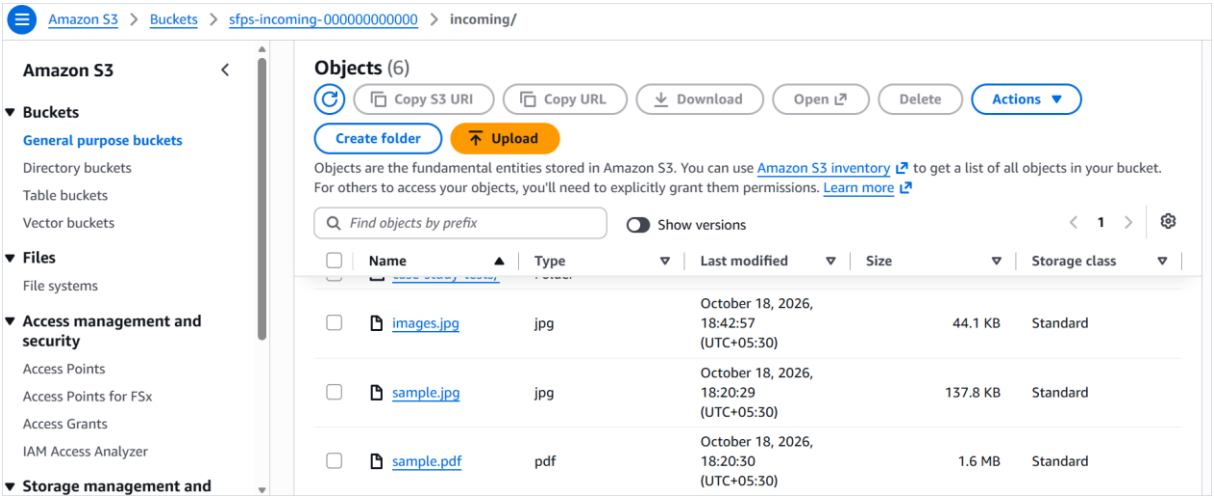


Figure 6.2. *Objects under incoming/, including sample.jpg, sample.png, sample.pdf, and images.jpg.*

The captured S3 pages do not show the Properties or Permissions tabs, so they do not visually prove versioning, encryption, public-access blocking, the TLS policy, or the lifecycle rule. Those settings are documented in the README and the console guide and are treated as implemented configuration, not as visual evidence. The bucket and prefix structure is visually verified.

6.3 Amazon DynamoDB table

The sfps-processing table uses processing_id as a String partition key, no sort key, on-demand capacity, AWS-owned encryption, and no point-in-time recovery. There are no secondary indexes. The table is the operational ledger for the pipeline. Figure 6.3 shows the table browser with the Scan view selected and all attributes projected.

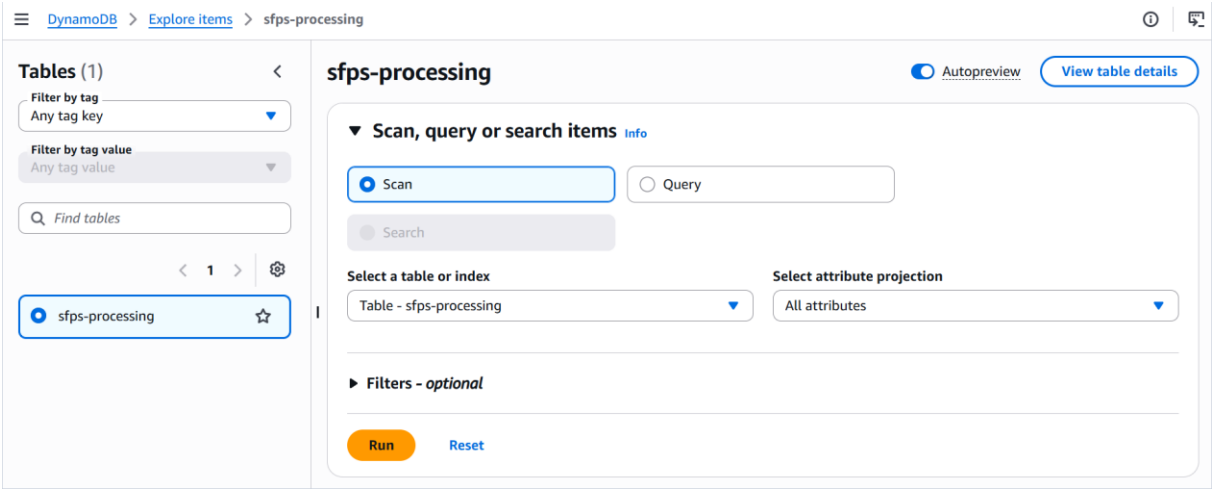


Figure 6.3. *DynamoDB table browser for sfps-processing with the Scan view selected.*

On-demand capacity was chosen because the workload is bursty and the project has no traffic forecast. The table does not use a sort key because the processing identity is already unique per object version. The absence of PITR and secondary indexes is a V1 scope decision. It keeps the table simple and avoids extra cost, but it also means there is no historical recovery or alternate query path beyond a scan. Chapter 13 treats that as a limitation.

6.4 Amazon SQS failure queue

The sfps-failed-events queue is a standard queue with SSE-SQS encryption, a 14-day message retention period, a 30-second visibility timeout, and no dead-letter queue. No consumer runs in V1. The queue is configured as the Lambda function's on-failure destination, not as a classic DLQ. The execution role has sqs:SendMessage on the queue ARN, which the destination delivery requires.

The queue is intentionally not the primary error path for invalid files. A PDF upload or a corrupt image becomes a FAILED DynamoDB record and does not reach SQS. The queue is for events that could not be processed because of an infrastructure fault after retries. This separation keeps the queue signal meaningful: a message in sfps-failed-events means an operational fault, not an ordinary rejected upload.

6.5 IAM role and scoped policy

The execution role trusts lambda.amazonaws.com. The documented role name is sfps-lambda-role, with an inline policy named sfps-lambda-scoped-policy. The captured IAM page shows a role named sfps-lambda-scoped-policy, created on 18 October 2026, with a one-hour maximum session duration. The role page does not show the attached policy document. The policy below is the repository-documented policy and should be treated as the intended configuration, not as a transcription of the console capture.

Table 6.1. *Documented IAM policy statements.*

Sid	Actions	Resource	Purpose
S3ReadPinnedObjects	s3:GetObject, s3:GetObjectVersion	arn:aws:s3:::sfps-incoming-<account-id>/incoming/*	Read the pinned object and its metadata. HeadObject is authorized by these actions.
DynamoProcessingTable	dynamodb:GetItem, dynamodb:PutItem, dynamodb:UpdateItem	arn:aws:dynamodb:<region>:<account-id>:table/sfps-processing	Claim, read, and update the processing record.
FailureQueueSend	sqs:SendMessage	arn:aws:sqs:<region>:<account-id>:sfps-failed-events	Allow the asynchronous on-failure destination to deliver the invocation envelope.
FunctionLogs	logs:CreateLogStream, logs:PutLogEvents	arn:aws:logs:<region>:<account-id>:log-group:/aws/lambda/sfps-processor:*	Write application and platform logs to the pre-created log group.

The policy deliberately omits logs:CreateLogGroup. That forces the operator to pre-create the log group, which is a small deployment step and a meaningful least-privilege choice. It also omits s3:ListBucket, DynamoDB scan or delete actions, and any wildcard resource outside the prefix and log-group suffixes required by the design. The role cannot read objects outside incoming/, cannot write to another table, and cannot send to another queue.

6.6 Lambda function configuration

The function is named sfps-processor, uses the Python 3.13 runtime, runs on arm64, has 512 MB of memory, and has a 30-second timeout. The package type is Zip. The handler is handler.lambda_handler. The function has no layers. The captured function list shows the runtime and package type; the general configuration page shows the memory and timeout.

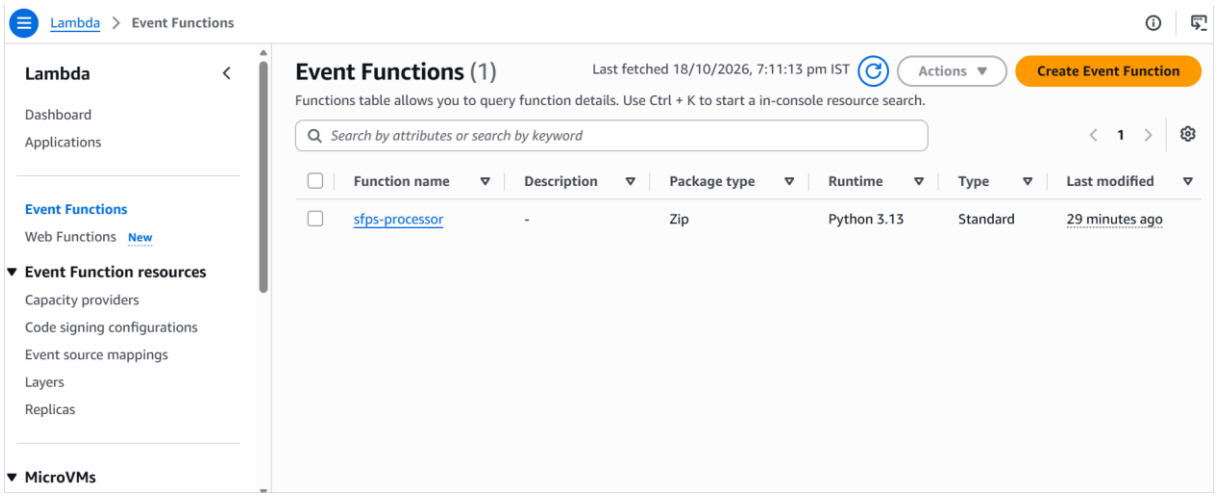


Figure 6.4. *Lambda function list showing sfps-processor as a Zip package on Python 3.13.*

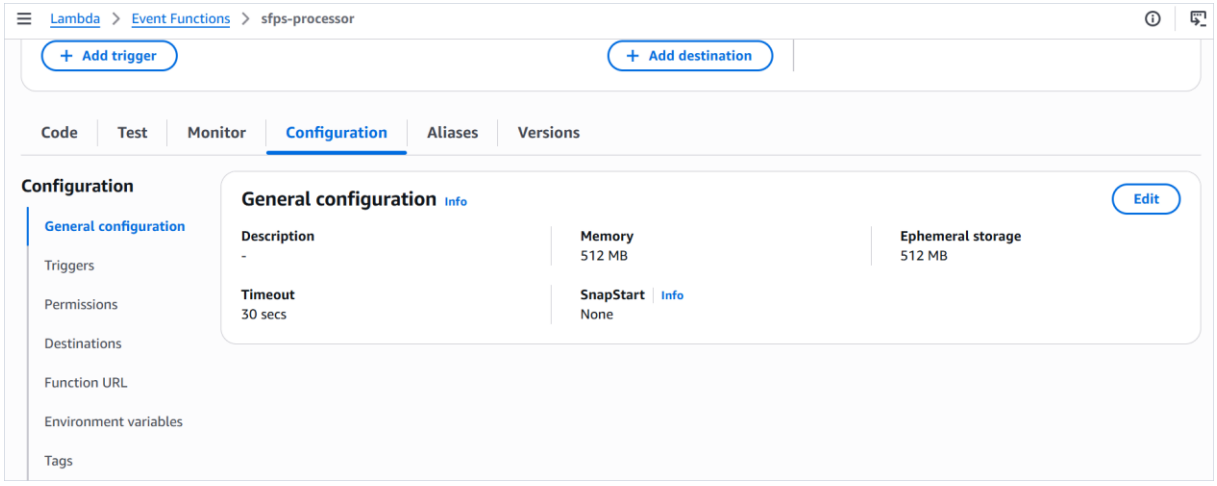


Figure 6.5. *Lambda general configuration showing 512 MB of memory and a 30-second timeout.*

The 512 MB memory setting is also the CPU allocation lever in Lambda. A higher memory setting can reduce duration for CPU-bound work, but it also raises the per-millisecond cost. The project chose 512 MB as a balance between image decoding needs and cost, without claiming that it is optimal. The 30-second timeout is well above the observed successful invocation durations, but the boundary input at 25 MiB was not measured on the deployed function.

6.7 Environment variables

The deployed environment has three variables: TABLE_NAME set to sfps-processing, MAX_FILE_SIZE_BYTES set to 26214400, and LOG_LEVEL set to INFO. The explicit values match the documented defaults and make the deployed configuration visible without reading code. The function validates the values at import and fails loudly if they are wrong.

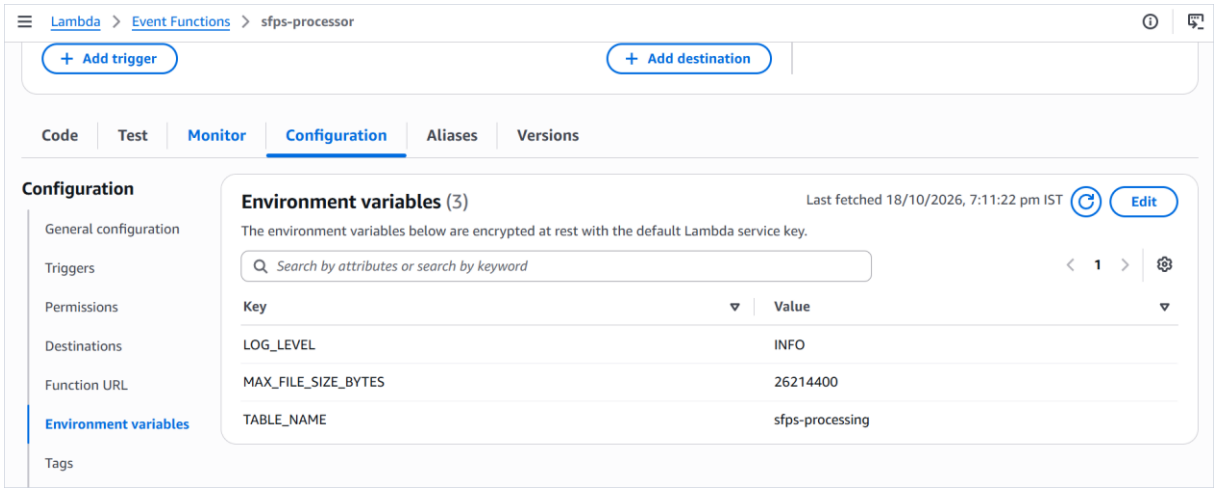


Figure 6.6. *Lambda environment variables for the deployed function.*

6.8 Asynchronous invocation and the SQS destination

The asynchronous invocation configuration sets MaximumRetryAttempts to 2 and leaves the maximum event age at the default six hours. The destination is an SQS destination with condition On failure and destination ARN arn:aws:sqs:ap-south-1:<account-id>:sfps-

failed-events. Figure 8.2 in the security chapter shows the destination page. The function overview in Figure 4.3 shows the same integration from the function's diagram view.

The destination is not a classic dead-letter queue. Lambda sends an invocation envelope with requestContext, requestPayload, and responseContext/responsePayload. requestPayload contains the entire original S3 event. The responsePayload of an exhausted invocation is sanitized by the handler boundary, which raises a fixed-message InfrastructureError from None. The captured envelope in Chapter 9 confirms that behavior.

6.9 S3 event trigger and invoke permission

The S3 trigger is configured with event types s3:ObjectCreated:*, prefix incoming/, and an empty suffix. The trigger details show the bucket ARN, the notification name, the service principal s3.amazonaws.com, the source account, and the statement ID. Creating the trigger from the Lambda console creates both the bucket notification and the resource-based invoke permission. The permission is scoped by source ARN and source account.

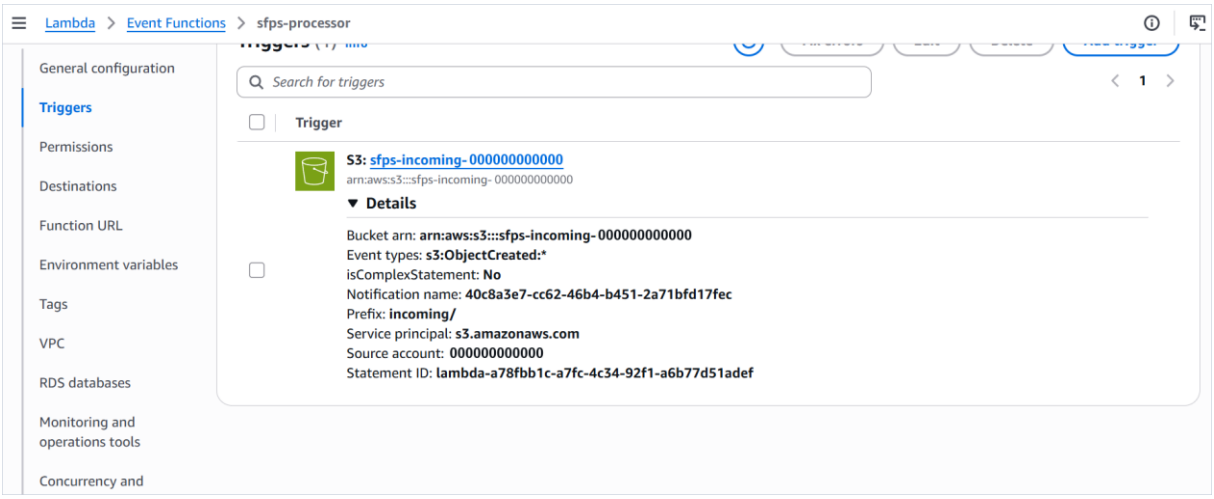


Figure 6.7. *S3 trigger details showing ObjectCreated:* events, the incoming/ prefix, and the source account.*

The empty suffix is important. A suffix filter for .jpg or .txt would prevent unsupported files from reaching Lambda, and the system would have no explicit record that they were rejected. With no suffix filter, a PDF or CSV reaches the function, fails validation, and produces a FAILED record. The queue remains reserved for infrastructure faults.

6.10 CloudWatch log group and retention

The log group is /aws/lambda/sfps-processor. It is pre-created because the execution role cannot create it. The documented retention is 30 days. The captured log group page shows the group and its Standard log class; the retention column is not visible in the capture. The function writes one JSON object per line, and the Lambda platform writes its own START, END, and REPORT lines. Figure 6.9 shows seven recent log streams, which is consistent with several execution environments over the test window.

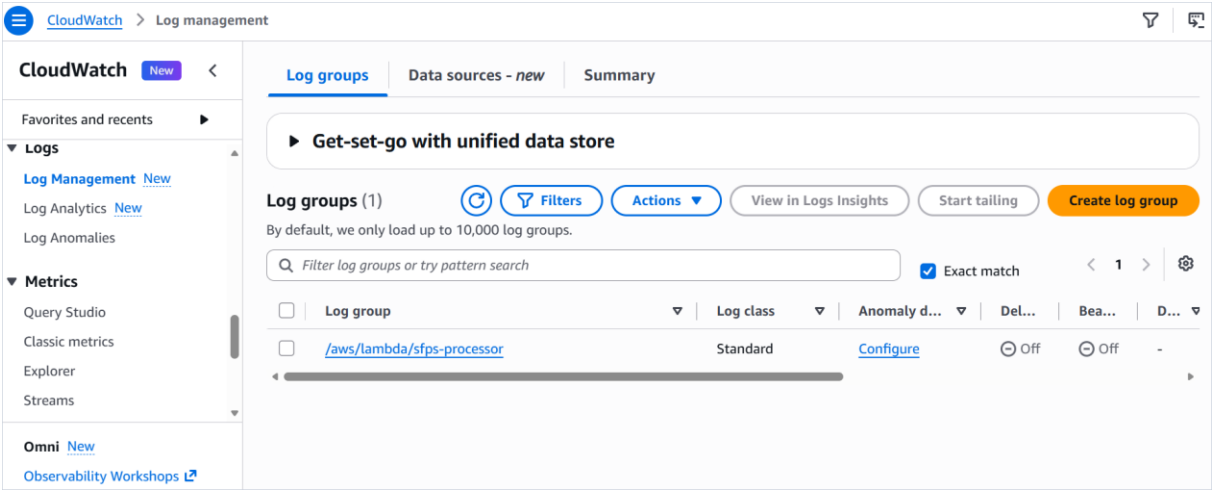


Figure 6.8. *CloudWatch log group for the processing function.*

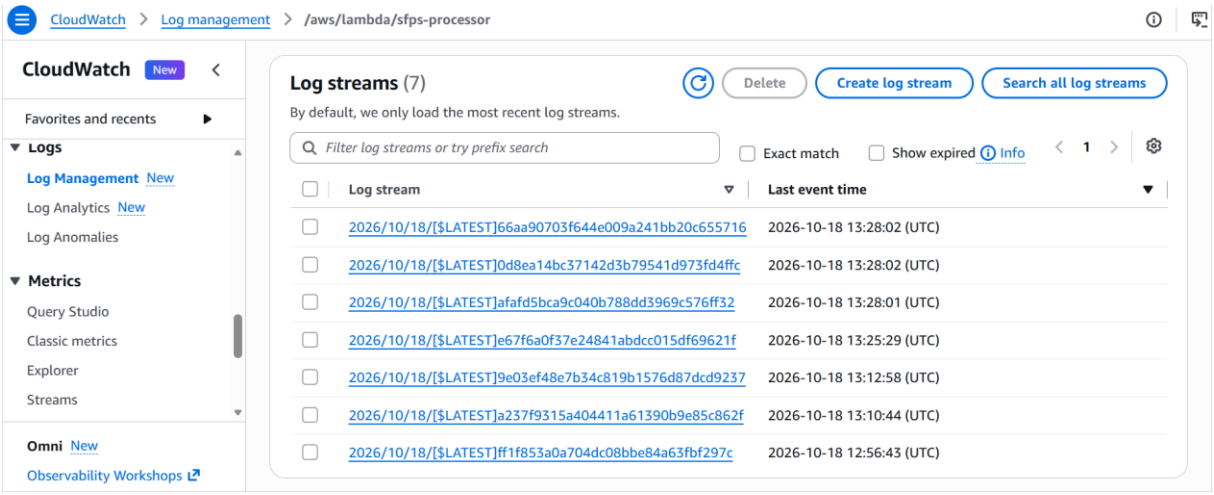


Figure 6.9. *Recent CloudWatch log streams for the processing function.*

6.11 Deployment evidence status

The console captures confirm that the bucket, prefix structure, DynamoDB table, Lambda function, environment variables, trigger, log group, IAM role page, and SQS destination exist. They do not confirm every configuration value. The evidence inventory in the appendix marks each item as visually verified, documented, or missing. This distinction is deliberate. A professional case study should not turn a console page that happens to be open into proof of a setting that is on another tab.

Chapter 7. Deployment and Packaging

7.1 Windows development environment

The repository is developed on Windows 11 with PowerShell 5.1 and Python 3.13.14. The Lambda target is Linux arm64. Those two facts create the main packaging problem: the build host cannot install the target's native wheels with a normal pip install, because pip would resolve wheels for the host platform. The project solves this with pip's cross-platform options: --python-version 3.13, --implementation cp, --abi cp313, --platform manylinux_2_28_aarch64, --only-binary=:all:, and --no-compile.

The virtual environment is created with python -m venv .venv and activated with .venv\Scripts\Activate.ps1. If PowerShell execution policy blocks activation, the same commands can be run with the explicit interpreter path .venv\Scripts\python.exe. The build does not depend on a globally installed Python package set.

7.2 Dependencies and the requirements lock

requirements.txt pins all eight runtime packages: boto3 1.43.112, botocore 1.43.112, jmespath 1.1.0, Pillow 12.3.0, python-dateutil 2.9.0.post0, s3transfer 0.19.2, six 1.17.0, and urllib3 2.8.0. requirements-dev.txt includes the runtime file and pins pytest 9.1.1, moto 5.2.4, and Ruff 0.17.0. Pinning every runtime transitive matters because the Lambda runtime also ships boto3; the project bundles its own pinned SDK so the deployed version does not drift with the runtime.

The build record found a previous package with botocore 1.43.113 while local tests used 1.43.112. That mismatch is exactly the kind of drift the lock is meant to prevent. The verified archive for this case study contains botocore 1.43.112 in its metadata, matching requirements.txt. The older workspace build/ directory still contains 1.43.113; it is a stale artifact and is not the archive used for deployment.

7.3 Why Windows-native binaries cannot be uploaded

A Python wheel is not always pure Python. Pillow ships compiled extension modules for image decoding, color management, fonts, and format support. On Windows those modules are .pyd files built for the Windows ABI. On Linux arm64 they are .so files built for the Linux ABI and the target CPU architecture. A Lambda function running on Linux arm64 cannot load a Windows .pyd file, and it cannot load an x86-64 .so file. The import would fail before the handler ran.

The same rule applies to any dependency with native code. Pure-Python packages can be packaged from any host, but binary packages must match the target's operating system, CPU architecture, Python ABI, and C library. The build command tells pip to download the target wheels instead of the host wheels, even though the download runs on Windows.

7.4 The manylinux_2_28_aarch64 platform tag

Lambda's Python 3.13 runtime runs on Amazon Linux 2023, which provides glibc 2.34. The manylinux_2_28 tag requires glibc 2.28 or newer, so it is compatible with the runtime. The PRD originally used manylinux2014_aarch64, which requires an older baseline. Pillow 12.3.0 does not publish a manylinux2014 aarch64 wheel for cp313; pip resolution for that tag stops at Pillow 12.2.0. Rather than downgrade Pillow or relax the lock, the project moved the build to manylinux_2_28_aarch64, which has a cp313 aarch64 wheel for the pinned version.

The --no-compile flag prevents pip from writing __pycache__ and .pyc files during the target install. The build also sweeps src/ for __pycache__ before copying. The archive verification checks that no bytecode or dev artifact is present. The older build/ directory is not used as evidence because it contains an older botocore version; the verified function.zip is the release artifact.

7.5 ZIP package layout

Lambda expects the handler module at the path named by the handler setting. The function is configured as handler.lambda_handler, so handler.py must sit at the ZIP root, not inside a src/ folder. The build copies the contents of src/ into build/, then zips the contents of build/ rather than the build/ directory itself. The ZIP entry names must use forward slashes. PowerShell 5.1's Compress-Archive writes backslash separators, which Lambda cannot unpack correctly, so the project uses Python's zipfile with os.sep replaced by a forward slash.

Code 7.1. *Build and verification commands used on the Windows host.*

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
Remove-Item -Recurse -Force build -ErrorAction SilentlyContinue
python -m pip install --python-version 3.13 --implementation cp --abi cp313 `
  --platform manylinux_2_28_aarch64 --only-binary=:all: --no-compile `
  --target build -r requirements.txt
Get-ChildItem -Path src -Recurse -Directory -Filter __pycache__ |
  Remove-Item -Recurse -Force
Copy-Item -Path src\* -Destination build\ -Recurse -Force
Remove-Item -Force function.zip -ErrorAction SilentlyContinue
python -c "import zipfile, os; z = zipfile.ZipFile('function.zip','w',zipfile.ZIP_DEFLATED); "
  "[z.write(os.path.join(dp,f), os.path.relpath(os.path.join(dp,f),'build').replace(os.sep,'/')) "
  "for dp,dn,fn in os.walk('build') for f in fn]; z.close()"
python -c "import zipfile; names = zipfile.ZipFile('function.zip').namelist(); "
  "assert 'handler.py' in names; assert all('\\' not in n for n in names); "
  "assert all(not n.startswith('./') for n in names); print(len(names), 'entries OK')"
```

The command in Code 7.1 is a readable line-wrapped form of the repository command. In PowerShell, the backtick is the line-continuation character, and the Python zipfile expression must remain on the logical line passed to python -c. The repository README contains the exact single-line commands used for the build.

7.6 Package verification results

Table 7.1. *Independent verification of the deployment archive.*

Check	Expected	Observed	Result
Archive exists	function.zip present	22,955,647 bytes compressed	Pass
Entry count	No fixed count; verify contents	2,358 entries	Pass
Uncompressed size	Under 250 MiB	45,828,329 bytes	Pass
Handler at root	handler.py at ZIP root	handler.py at root	Pass
Forward slashes	No backslash entry names	No backslashes found	Pass
No ./ prefix	No entry starts with ./	None found	Pass
No bytecode	No __pycache__ or .pyc	None found	Pass
ARM64 extensions	aarch64 .so files present	8 Pillow .so files, aarch64-linux-gnu	Pass
Runtime pins	All 8 versions match requirements.txt	boto3 1.43.112, botocore 1.43.112, jmespath 1.1.0, Pillow 12.3.0, python-dateutil 2.9.0.post0, s3transfer 0.19.2, six 1.17.0, urllib3 2.8.0	Pass
Direct-upload limit	Under 50 MiB	22,955,647 bytes	Pass

The verification was repeated for this case study by inspecting the existing function.zip. The archive is below the 50 MB direct-upload limit, so no S3-hosted upload was necessary. The presence of aarch64 .so files is strong evidence that the package contains Linux arm64 binaries rather than Windows binaries. It is not proof that every native extension imports correctly at runtime; the deployed invocations provide that evidence for the code paths they exercised.

7.7 Upload and runtime configuration

The package is uploaded through the Lambda console (Code, Upload from, .zip file) or with the AWS CLI: aws lambda update-function-code --function-name sfps-processor --zip-file fileb://function.zip --region ap-south-1. After upload, the handler is set to handler.lambda_handler, the runtime to Python 3.13, the architecture to arm64, the memory to 512 MB, and the timeout to 30 seconds. The environment variables are set as shown in Chapter 6. The asynchronous invocation configuration and the SQS destination are configured separately.

The deployment guide recommends verifying the last update status before testing. It also warns that the package is larger than the embedded editor's 3 MB threshold, so the source tree may not be visible in the console even when the upload succeeded. That warning prevents a false diagnosis: an empty editor is not evidence that the package failed to upload.

7.8 Deployment challenges and solutions

The first packaging attempt used the PRD's original manylinux2014_aarch64 platform tag. Pip could not resolve Pillow 12.3.0 for cp313 under that tag; it offered at most 12.2.0. The project could have downgraded Pillow, but the dependency pins were treated as fixed. The resolution was to change the platform tag to manylinux_2_28_aarch64 and document the reason in ADR-012. This is a useful example of a build constraint that is invisible until the exact combination of Python version, library version, and platform tag is tested.

The second challenge was bytecode in the package. Pip's --target install byte-compiles by default, which produced __pycache__ and .pyc files that violated the package's no-bytecode check. The --no-compile flag prevents the problem at the source, and a __pycache__ sweep remains as a guard. The third challenge was ZIP entry separators. Compress-Archive produced backslash names that Lambda could not unpack. The Python zipfile command with an explicit separator replacement fixed it, and the verification assert catches a regression before upload.

A fourth challenge is evidence drift. The workspace build/ directory contains an older botocore 1.43.113, while the release archive contains 1.43.112. A casual inspection of build/ would therefore describe the wrong package. The project treats function.zip as the release artifact and compares its installed metadata against requirements.txt. The stale build directory is left in place but is not cited as evidence. This is a small example of a larger lesson: a build directory is not a release record.

Chapter 8. Security and Reliability

8.1 Security model and boundaries

SFPS treats the uploaded file as untrusted input and the AWS account as the trust boundary. The bucket is private, the function has a narrow execution role, and the object content never leaves the AWS services that need it. The function reads bytes into memory, validates them without executing them, and writes only derived metadata, a checksum, and a small error map to DynamoDB. It does not write files to disk, spawn subprocesses, or call external services.

The security review looks at four surfaces: the storage boundary, the compute boundary, the identity boundary, and the diagnostic boundary. The storage boundary covers public access, TLS, encryption, and versioning. The compute boundary covers input validation, resource limits, and the absence of code execution. The identity boundary covers IAM scope and resource-level restrictions. The diagnostic boundary covers log fields, error messages, and destination payloads.

8.2 Private S3 access and TLS

Block all public access is enabled on the bucket. The bucket policy denies every s3:* request whose aws:SecureTransport condition is false. This is a deny-only policy with Principal "*"; it does not grant public access. The console may warn about the wildcard principal, but the statement only denies non-TLS requests. The function's IAM policy grants read access only to the incoming/ prefix, so even a compromised function could not read objects elsewhere in the bucket.

The project does not use presigned URLs, a public website endpoint, or an S3 static website configuration. Uploads are performed by an authenticated principal through the console or the AWS CLI over TLS. The function never writes to S3, so there is no write path to protect beyond the uploader's own permissions.

8.3 Encryption

Default encryption is SSE-S3, which encrypts each object at rest with Amazon S3 managed keys. S3 Bucket Keys apply to SSE-KMS, not SSE-S3, so there is no bucket-key setting to configure for this design. The SQS queue uses SSE-SQS by default. DynamoDB uses AWS-owned encryption. These choices meet the V1 requirement to encrypt data at rest without adding a customer-managed KMS key and its key-policy surface. A KMS customer-managed key is a reasonable future enhancement for organizations that need key rotation control or cross-account key policies.

8.4 IAM least privilege

The execution role trusts only lambda.amazonaws.com. The inline policy grants four groups of actions. S3 read access is limited to the bucket's incoming/ prefix. DynamoDB access is limited to GetItem, PutItem, and UpdateItem on the processing table. SQS access is limited to SendMessage on the failure queue. CloudWatch access is limited to CreateLogStream and PutLogEvents on the function's log group. There is no s3:ListBucket, no DynamoDB Scan, no DeleteItem, and no wildcard resource outside the required prefix and log-group suffix.

The role deliberately lacks logs:CreateLogGroup, which is why the log group is pre-created. This is a small operational inconvenience and a meaningful reduction in scope. The role cannot create new log groups, change retention, or delete logs. It can only write to the group that the deployment created.

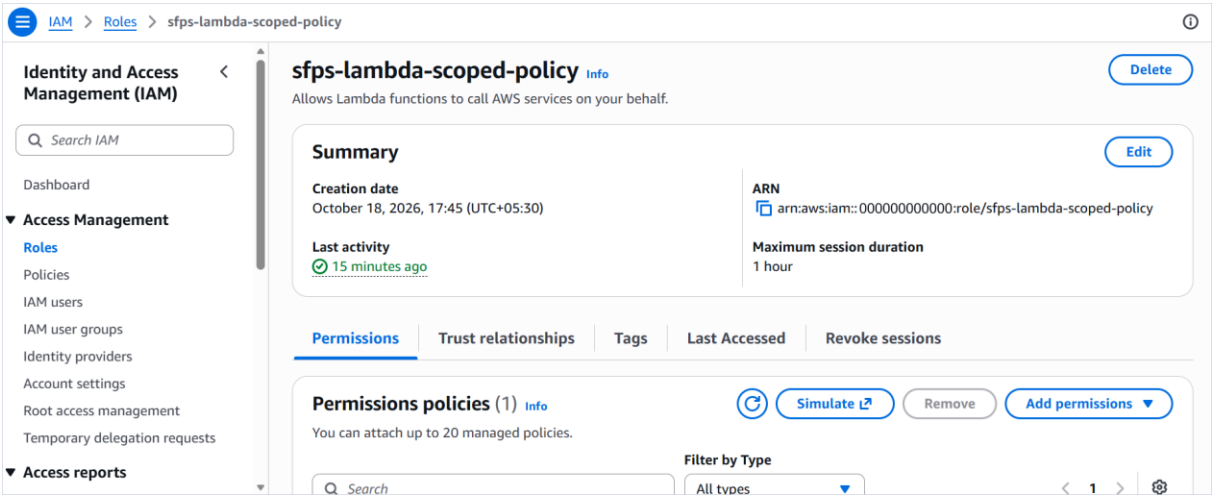


Figure 8.1. *IAM execution role page captured for the deployed function.*

The captured role page is named sfps-lambda-scoped-policy, while the repository documentation uses sfps-lambda-role for the role and sfps-lambda-scoped-policy for the inline policy. The difference is recorded here rather than smoothed over. The page confirms that a role with the policy name exists and was created on 18 October 2026. It does not show the attached policy document, so the action-level scope is verified from the repository policy and the deployment guide, not from this capture.

8.5 Resource-scoped permissions

Resource scoping is the part of least privilege that is easiest to get wrong. The S3 statement uses arn:aws:s3:::sfps-incoming-<account-id>/incoming/*, not the bucket ARN alone. The DynamoDB statement uses the table ARN, not a wildcard table pattern. The SQS statement uses the queue ARN. The CloudWatch statement uses the log-group ARN with the :* suffix required for log streams. These are deliberately scoped wildcards, not broad grants. The deployment guide explicitly tells the operator not to add s3:ListBucket, DynamoDB scan/delete, or administrator policies to the execution role.

The Lambda resource-based policy is a separate boundary. It allows s3.amazonaws.com to invoke the function, scoped by AWS:SourceArn to the bucket and AWS:SourceAccount to the account. The trigger details in Figure 6.7 show the service principal and source account. If an older console creates the statement without the source conditions, the guide provides CLI commands to remove and re-add it with the restriction.

8.6 Version-pinned processing

Version pinning is a reliability control as well as a correctness control. If an object is overwritten while an event is being processed, an unpinned read could fetch the new bytes while the record still refers to the old event. SFPS passes the event's version ID to both HeadObject and GetObject, so the size and bytes belong to the same version. A missing or null version ID is a malformed event, not a cue to read the latest version. This avoids a silent mismatch between the recorded identity and the processed content.

The lifecycle rule that expires noncurrent versions after 30 days bounds storage growth. It does not remove the current version. A version that is still current remains readable even after its record is completed. If an operator needs to investigate an old event after the noncurrent window, the bytes may no longer exist, but the DynamoDB record and the CloudWatch logs remain. That is a deliberate retention trade-off, not an oversight.

8.7 DynamoDB state tracking and conditional writes

The table records PROCESSING, COMPLETED, and FAILED states. The claim is conditional, so two concurrent invocations cannot both create the same processing_id. The terminal update is conditional on status <> COMPLETED, so a late or duplicate invocation cannot overwrite a completed result. The duplicate path uses ConsistentRead=True after a claim conflict, which prevents a stale replica from causing unnecessary reprocessing. These controls are tested with duplicate delivery, a PROCESSING re-attempt, and a simulated terminal race.

The table does not use transactions across records. Each record is independent, which matches the per-version processing model. A mixed batch can leave earlier records COMPLETED while a later record fails. That is expected. The batch is not a unit of atomicity; the object version is.

8.8 Retry and failure destination

Lambda asynchronous invocation retries function errors up to two times after the initial attempt. Throttling and system errors have separate retry behavior up to the maximum event age. The on-failure destination is SQS, not a classic DLQ. The execution role needs sqs:SendMessage on the queue for the destination to work. The queue has no consumer in V1, so a message stays until the 14-day retention period expires unless an operator deletes it.

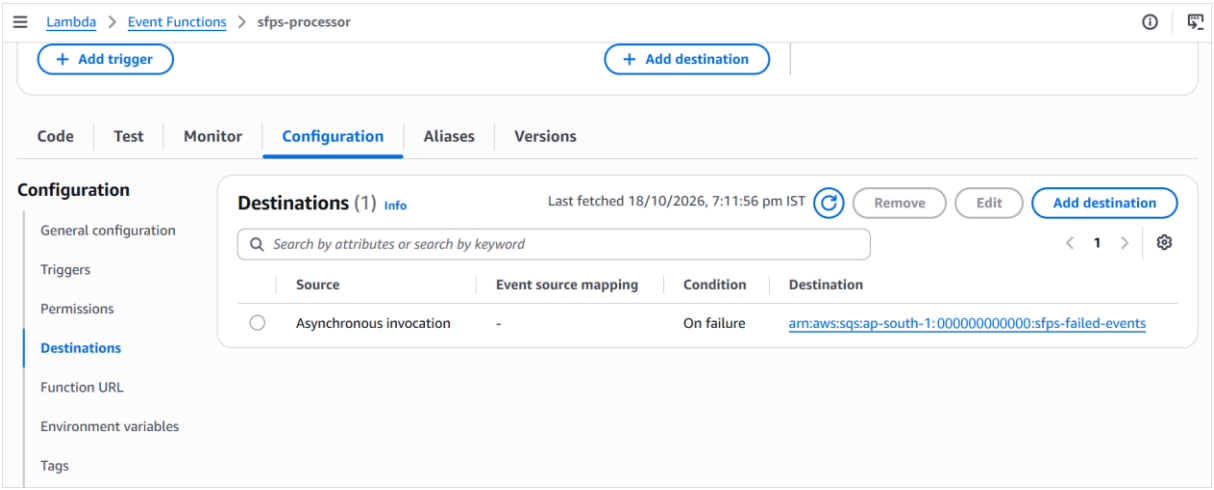


Figure 8.2. *Lambda asynchronous invocation destination configured for On failure to the SQS queue.*

A permanent validation failure does not itself trigger retries or destination delivery. A mixed batch can still be retried and delivered if another record in the same event fails with an infrastructure error. The destination envelope then contains every original record. This behavior is documented in the PRD, the README, and the deployment guide because it is easy to misread as a failed validation sending itself to SQS.

8.9 Structured logs and sanitized diagnostics

The application logger writes one JSON object per line. The formatter includes a fixed set of fields and omits absent values. Stack traces include frame filenames, function names, and line numbers, and omit exception text, chained causes, source lines, and local variables. Persisted error messages come from an allowlist, and unexpected exceptions become a fixed infrastructure message. The outer Lambda boundary raises a fixed-message InfrastructureError from None, so raw SDK or decoder text does not appear in the runtime log or the destination responsePayload.

The regression suite exercises these surfaces with a marker string that must not appear in the persisted error, the emitted JSON, or the rendered uncaught exception. The test includes a chained cause, because exception chaining is a common leak path. The logger's propagate flag is False, and a test installs a root handler to confirm that application records do not reach it.

8.10 Duplicate event handling

S3 event notifications are at-least-once. Duplicate delivery of the same bucket, key, and version is possible. The handler skips only COMPLETED records. A PROCESSING record is re-attempted, because the previous attempt may have crashed before the terminal write. A FAILED record is re-attempted, because a later delivery may succeed after a transient fault. The created_at timestamp is never overwritten. The terminal update cannot overwrite COMPLETED. Together these rules make retries converge to the true terminal state.

The duplicate path performs no S3 fetch when the record is already COMPLETED. A regression test simulates a stale read and a forbidden HeadObject call, so a future change that reintroduces the fetch fails the test. This matters for cost and latency as well as correctness: duplicate notifications should not re-download objects.

8.11 Data retention

S3 noncurrent versions expire after 30 days. CloudWatch log retention is 30 days. SQS messages are retained for 14 days. DynamoDB records have no TTL and remain until deleted. The error map is capped at 1,024 serialized characters. The function does not store file contents in DynamoDB or CloudWatch. The only copy of the original bytes is the S3 object version.

The asymmetry is intentional. Processing records are the operational ledger and are kept for inspection. Source objects are retained as current versions and for 30 days as noncurrent versions. Logs and failure messages have shorter windows because they are diagnostic, not the system of record. A future retention policy could add a DynamoDB TTL or an archive export, but V1 does not.

8.12 Reliability limitations and operational risks

The design has real limitations. There is no automated alarm when a message lands in SQS. There is no consumer that replays failed events. There is no dead-letter queue for the SQS queue itself. There is no cross-region replication. The DynamoDB table has no point-in-time recovery. The function has no reserved concurrency, so a sudden burst can be throttled. The 25 MiB memory and timeout boundary has not been measured on the deployed function. The IAM policy attachment is documented but not visually verified. The log retention value is documented but not visible in the capture.

These are not reasons to call the system unreliable for its V1 purpose. They are reasons not to call it production-grade without additional work. A production deployment would add alarms, a replay procedure, load and boundary tests, a recovery plan, and probably infrastructure as code. Chapter 13 lists those enhancements and separates them from what exists today.

Table 8.1. *Security controls and evidence status.*

Control	Implementation	Evidence	Status
Private bucket	Block all public access enabled.	Documented in README and deployment guide.	Implemented; not visible in capture.
TLS-only access	Bucket policy denies aws:SecureTransport false.	Policy JSON in README and guide.	Implemented; not visible in capture.
Encryption at rest	SSE-S3 for S3; SSE-SQS for SQS; AWS-owned for DynamoDB.	Documented configuration.	Implemented; not visible in capture.
Least-privilege role	Scoped inline policy for S3 prefix, table, queue, and log group.	Policy documented; role page captured.	Implemented; action-level scope not visible in capture.
Version-pinned reads	VersionId on HeadObject and GetObject.	Automated version-pinning test.	Verified.
Conditional writes	Conditional PutItem and UpdateItem.	Automated duplicate, race, and re-attempt tests.	Verified.
Sanitized diagnostics	Allowlisted messages; sanitized frames; fixed boundary error.	Regression tests with marker strings and chained causes.	Verified.
Retry and destination	MaximumRetryAttempts=2; SQS on-failure destination.	Destination capture; SQS envelope with RetriesExhausted and invoke count 3.	Verified for one injected failure.
Log retention	30 days on the function log group.	Documented; log group capture does not show retention.	Implemented; not independently verified.
No credential exposure	No secrets in code or logs; IAM role used at runtime.	Screenshots and repository review show no access keys or session tokens.	Verified by inspection of provided evidence.

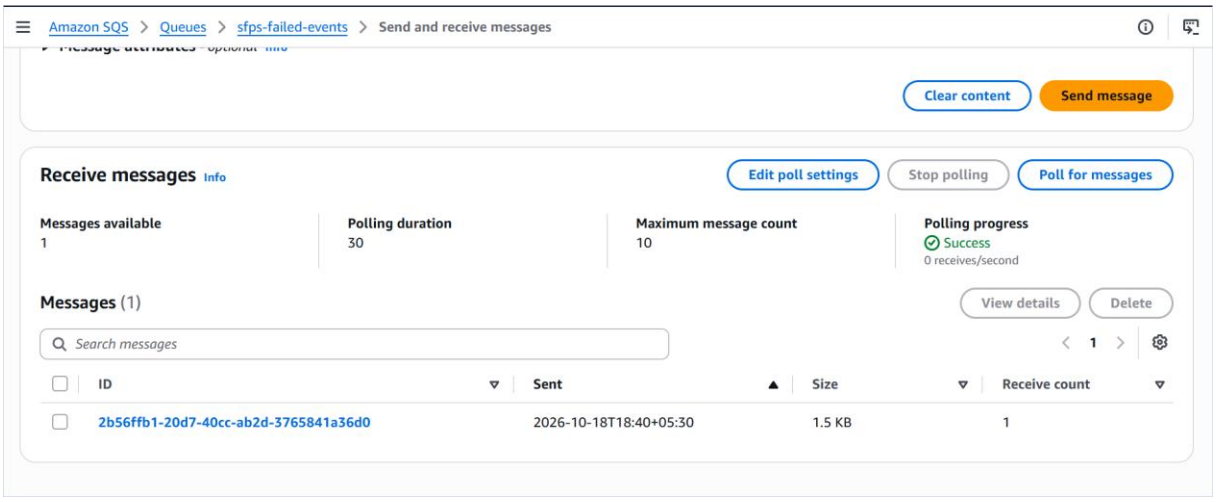


Figure 8.3. *SQS queue with one received message after an intentionally triggered infrastructure failure.*

8.13 Claims not made

The project does not claim that the bucket configuration is production-grade for regulated data, that the IAM policy has been penetration-tested, that the function is immune to decoder vulnerabilities, or that the queue is monitored. It does not claim multi-region durability, zero data loss, or a recovery point objective. It does not claim that the deployed function has been load-tested. The security and reliability claims are limited to the controls that are implemented and the evidence that is shown.

Chapter 9. Testing Strategy and Test Execution

9.1 Testing approach

The testing strategy uses three layers. The first layer is a pure unit suite for validation, checksum, image metadata, and text metadata. These tests need no AWS and no network. The second layer is a handler suite with moto-mocked S3 and DynamoDB. It invokes lambda_handler with synthetic S3 events, verifies records, and exercises duplicate delivery, malformed events, conditional writes, transient errors, and log behavior. The third layer is the deployed acceptance run documented by the console captures, the DynamoDB scan, the CloudWatch log views, and the SQS envelope.

The layers answer different questions. The unit and handler suites answer whether the repository implements the intended logic. The deployed acceptance run answers whether the resources were configured and whether specific inputs produced specific records. Neither layer substitutes for the other. A moto test passing does not prove that the deployed IAM policy works, and a console screenshot does not prove that an edge case in the validation order is correct.

9.2 Environment and commands

The local environment is Windows 11, Python 3.13.14, pytest 9.1.1, moto 5.2.4, and Ruff 0.17.0. The test command is python -m pytest -q, and the lint command is python -m ruff check src tests. The dependency check is python -m pip check. The handler tests set dummy AWS credentials and a region in tests/conftest.py before collection, then reload handler inside mock_aws() so the boto3 clients created at import are intercepted.

The suite was re-run for this case study on 18 October 2026. It collected 135 tests and reported 135 passed in 48.66 seconds. Ruff reported all checks passed. pip check reported no broken requirements. The 16 October build record reported 135 passed in 49.34 seconds, so the result is consistent with the recorded build state. The small timing difference is ordinary machine variation.

Table 9.1. *Collected test distribution.*

Test file	Collected tests	Main coverage
tests/test_handler.py	55	Configuration, event parsing, identity, claim protocol, idempotency, success pipeline, pre-download failures, version pinning, content failures, transient errors, structured logging.
tests/test_validation.py	26	Extension extraction and allowlist, image signatures, truncation, type mismatch, UTF-8 decoding, BOM handling.
tests/test_audit_regressions.py	39	PNG checksum damage, unexpected decoder errors, pixel budget, strong duplicate read, logger propagation, identity types, error redaction, text memory bounds.
tests/test_text.py	7	Character, word, and line counts; trailing newlines; whitespace; Unicode; CRLF; exact key set.
tests/test_image.py	5	Image dimensions and format; JPEG and PNG; exact key set; corrupt bytes.
tests/test_checksum.py	3	Empty digest, known abc digest, lowercase 64-character output.
Total	135	All tests passed on 18 October 2026.

9.3 Functional testing

Functional testing covered the happy path for each supported type: JPEG, PNG, and UTF-8 text. The handler tests build a versioned object in moto S3, invoke the handler with a synthetic event, and read the resulting DynamoDB item. The assertions check the record identity, status, file name, file type, file size, SHA-256, metadata, timestamps, and the absence of an error map on success. The deployed acceptance run produced matching COMPLETED records for sample.jpg, sample.png, sample.txt, and images.jpg.

The functional tests also cover uppercase extensions, nested keys, URL-encoded keys, and a multi-record batch with a permanent failure and a success. The URL decoding tests include %20, a literal plus, and %2B, which confirms that unquote_plus behaves as intended. The nested-key tests cover incoming/2026/photo.jpg and incoming/users/demo/file.txt. The deployed positive suite adds 06-nested-image.jpg in a nested/ folder.

9.4 Positive file-validation testing

The deployed positive suite contains six files: 01-landscape.jpg, 02-portrait.jpeg, 03-square.png, 04-multiline.txt, 05-unicode.txt, and 06-nested-image.jpg. The owner reports that all six completed successfully. The DynamoDB scan corroborates the status for each file: the rows show no error map and the file names match. Figure 9.3 shows the positive folder listing, including the nested/ folder. Figures 9.4 to 9.7 show the corresponding completed records.

The initial acceptance set adds sample.jpg, sample.png, and sample.txt. The scan contains COMPLETED records for all three, plus images.jpg. The sample.png file appears twice because a second version was uploaded for the oversized-dimension test; the smaller version completed and the larger version failed. This is the versioning behavior described in Chapter 4, not a duplicate processing bug.

9.5 Negative file-validation testing

The deployed negative suite contains six files: 07-empty.txt, 08-unsupported.csv, 09-invalid-utf8.txt, 10-fake-jpeg.jpg, 11-corrupt-png.png, and 12-over-pixel-limit.png. The owner reports that each produced its expected validation error. The DynamoDB scan corroborates the FAILED status and the presence of an error map for each row. The console table truncates the error value, so the exact error_code is not legible in the screenshots. The expected codes follow directly from the validation order: ZERO_BYTE, UNSUPPORTED_EXTENSION, INVALID_ENCODING, TYPE_MISMATCH, CORRUPT_IMAGE, and IMAGE_DIMENSIONS_EXCEEDED.

The initial negative set adds sample.pdf and the zero-byte folder markers. The PDF is 1.6 MB and is rejected before download with UNSUPPORTED_EXTENSION. The folder markers are zero-byte objects created when the console folders were made; they fail with

ZERO_BYTE. The scan shows seven such records with an empty file name. They are a useful bonus test of the zero-byte path, and they explain a large part of the failed-record count.

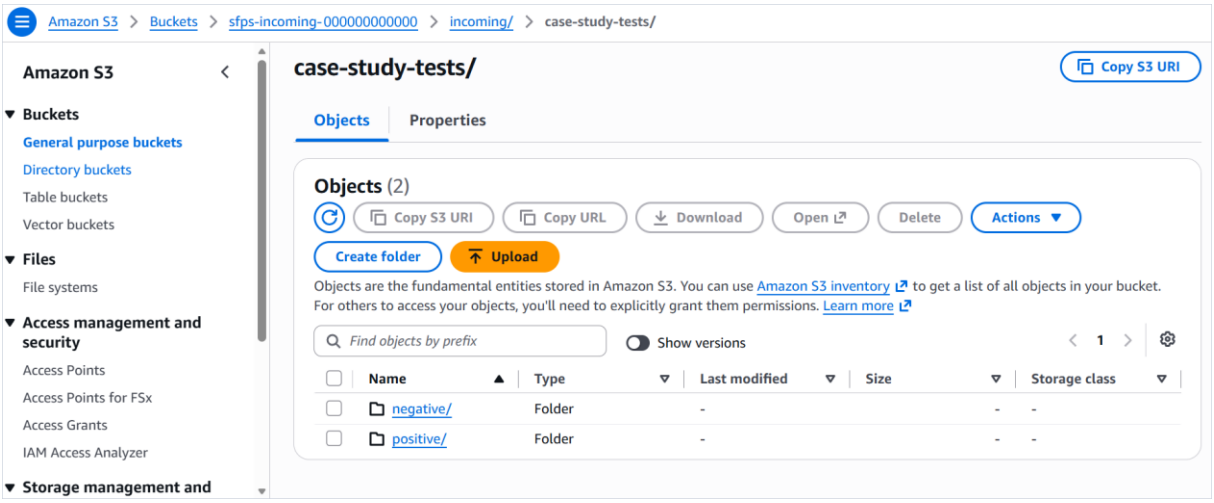


Figure 9.1. *S3 test-fixture structure under incoming/case-study-tests/ with separate positive and negative folders.*

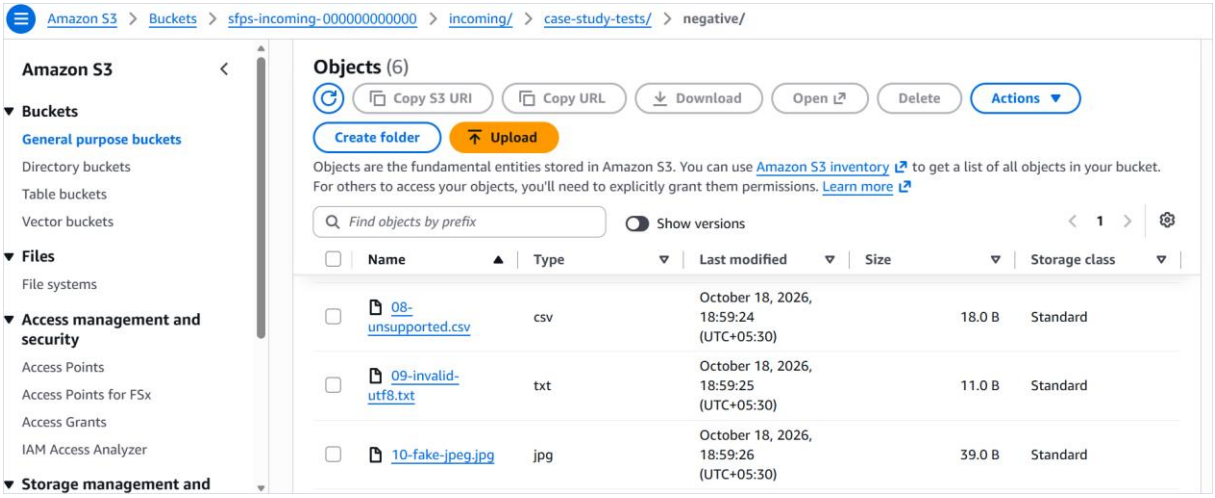


Figure 9.2. *Negative test fixtures, including 08-unsupported.csv, 09-invalid-utf8.txt, and 10-fake-jpeg.jpg.*

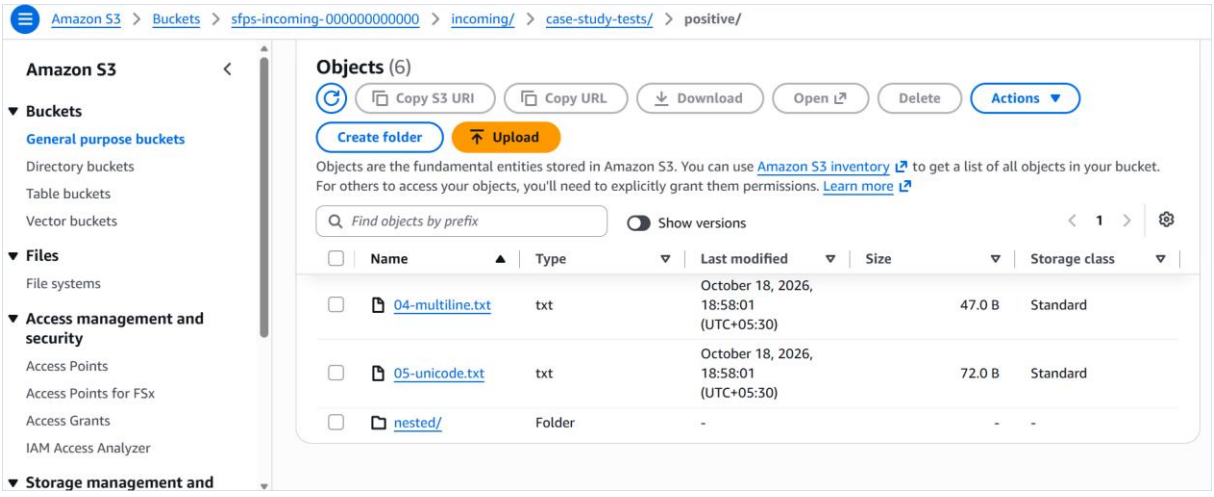


Figure 9.3. *Positive test fixtures, including 04-multiline.txt, 05-unicode.txt, and the nested/ folder.*

9.6 Nested-path testing

Nested-path support is covered at three levels. The unit-level handler test invokes incoming/2026/photo.jpg and incoming/users/demo/file.txt. The deployed positive suite includes a nested/ folder and a 06-nested-image.jpg record that completed. The S3 trigger uses the incoming/ prefix, which applies to all nested keys under it. The scan row for 06-nested-image has no error map, so the deployed path handling worked for that file. The exact full key is not visible in the truncated table, which is why the document refers to the nested folder and the file name rather than asserting the complete key string.

9.7 Error-handling testing

The handler suite covers the error paths that matter for correct operation. A malformed event with a missing or null version ID raises and writes nothing. A missing bucket or key raises and writes nothing. A non-string identity field raises and writes nothing, including numeric, boolean, list, object, and null values. A claim failure raises an InfrastructureError and writes nothing. A HeadObject service error writes FAILED with INFRASTRUCTURE and raises. A terminal COMPLETED update failure writes FAILED and raises. A secondary failure is logged and the safe exception is raised. An unexpected image processor exception writes FAILED with INFRASTRUCTURE and does not leak the raw message. A multi-record batch stops at the first infrastructure failure and leaves earlier COMPLETED records intact.

The error truncation tests use a 5,000-character validation message and a long untrusted message with quotes and backslashes. The serialized error must stay within 1,024 characters, the error_code and error_type must remain intact, and the message must be truncated rather than the classification. The redaction tests use a marker string and a chained cause; the marker must not appear in the persisted error, the emitted JSON, or the rendered uncaught exception.

9.8 DynamoDB verification

The deployed DynamoDB scan returned 25 items and scanned 25 items with 100 percent efficiency and 2.5 RCUs consumed. The item count is the first thing to interpret correctly. It is not a count of 25 independent test scenarios. The 25 records comprise 10 COMPLETED and 15 FAILED records, and several records belong to the same logical test. Table 9.2 breaks down the composition.

Table 9.2. *Composition of the 25-item DynamoDB scan.*

Category	Count	Records in the scan
COMPLETED: initial acceptance	4	sample.jpg, sample.png (smaller version), sample.txt, images.jpg.
COMPLETED: additional positive suite	6	01-landscape.jpg, 02-portrait.jpeg, 03-square.png, 04-multiline.txt, 05-unicode.txt, 06-nested-image.jpg.
FAILED: initial negative	2	sample.pdf (UNSUPPORTED_EXTENSION), sample.png larger version (oversized dimensions).
FAILED: zero-byte folder markers	7	incoming/ and nested folder markers created by the S3 console; empty file name, 0 bytes, error map present.
FAILED: additional negative suite	6	07-empty.txt, 08-unsupported.csv, 09-invalid-utf8.txt, 10-fake-jpeg.jpg, 11-corrupt-png.png, 12-over-pixel-limit.png.
Total	25	10 COMPLETED and 15 FAILED. The count includes two versions of sample.png and seven folder markers.

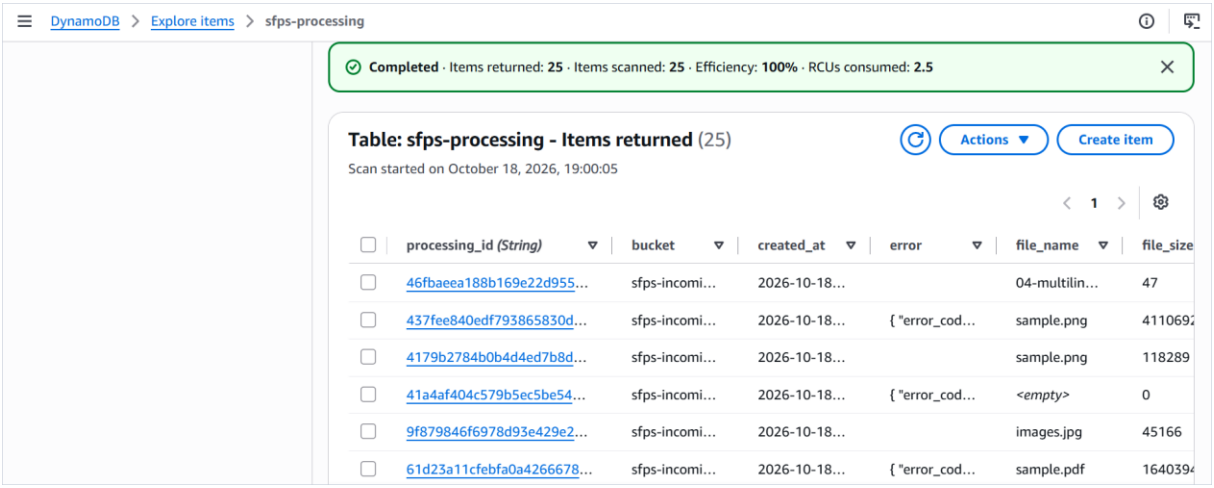


Figure 9.4. *DynamoDB scan summary showing 25 items returned, including completed sample records and failed versions.*

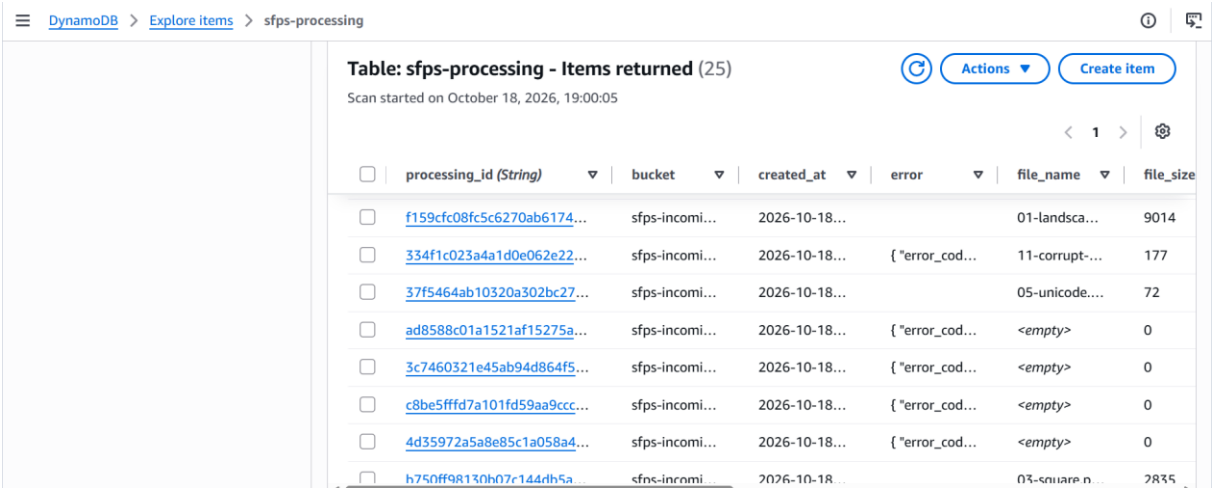


Figure 9.5. *DynamoDB rows for the additional positive and negative fixtures, including 01-landscape.jpg, 11-corrupt-png.png, and 05-unicode.txt.*

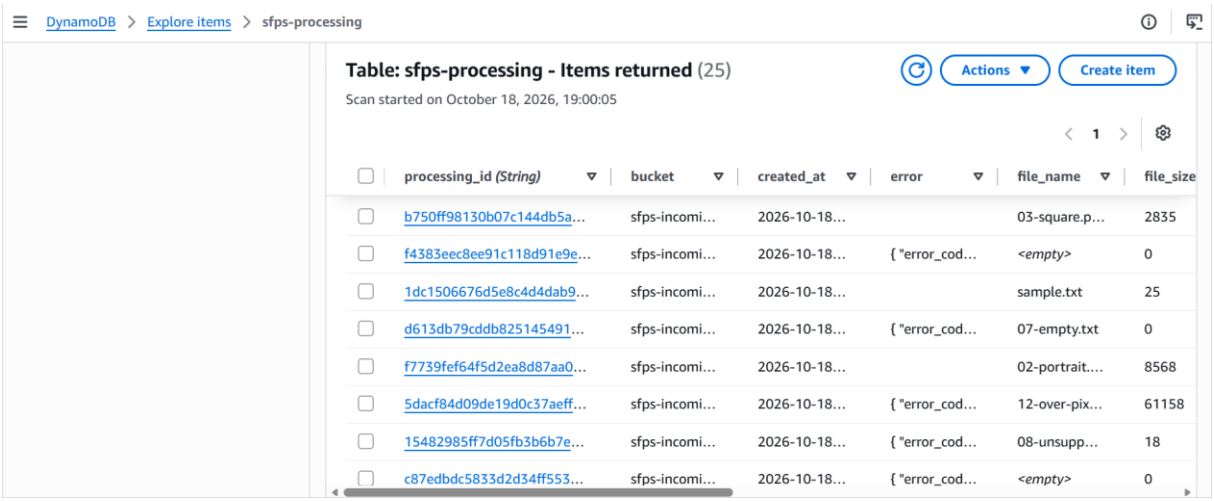


Figure 9.6. *Additional DynamoDB rows including 03-square.png, sample.txt, 02-portrait.jpeg, 12-over-pixel-limit.png, and 08-unsupported.csv.*

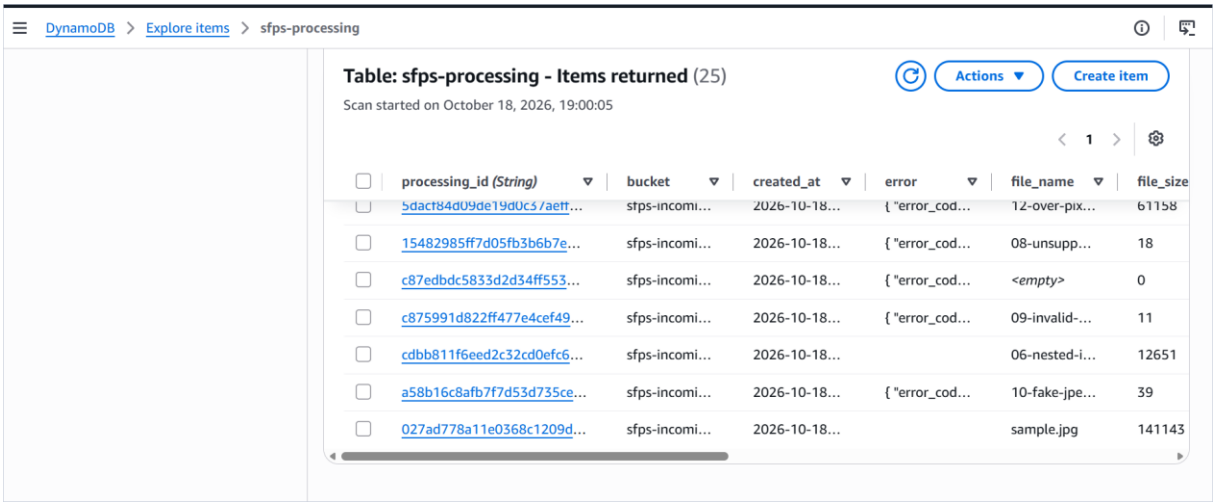


Figure 9.7. *Final DynamoDB rows including 09-invalid-utf8.txt, 06-nested-image.jpg, 10-fake-jpeg.jpg, and sample.jpg.*

The scan rows show the processing_id, bucket, created_at, error, file_name, and file_size columns. The error column is truncated, so the screenshots prove FAILED status and an error map rather than the exact code. The timestamps show that the records cluster on 18 October 2026, which matches the test window. The table does not show metadata, sha256, or processed_at in these column views, so those attributes are verified by the automated tests and the record schema, not by the screenshots.

9.9 CloudWatch verification

The CloudWatch log group exists and contains seven recent log streams. The log event views show the Lambda platform START, END, and REPORT lines plus application JSON lines. The visible application lines include a PROCESSING start, a permanent validation failure terminal line, and the corresponding END and REPORT lines. The JSON field values are truncated by the console width, so the capture

proves the line format and the status values rather than every field value. The log group page does not show the retention setting; 30-day retention is documented but not visually verified.

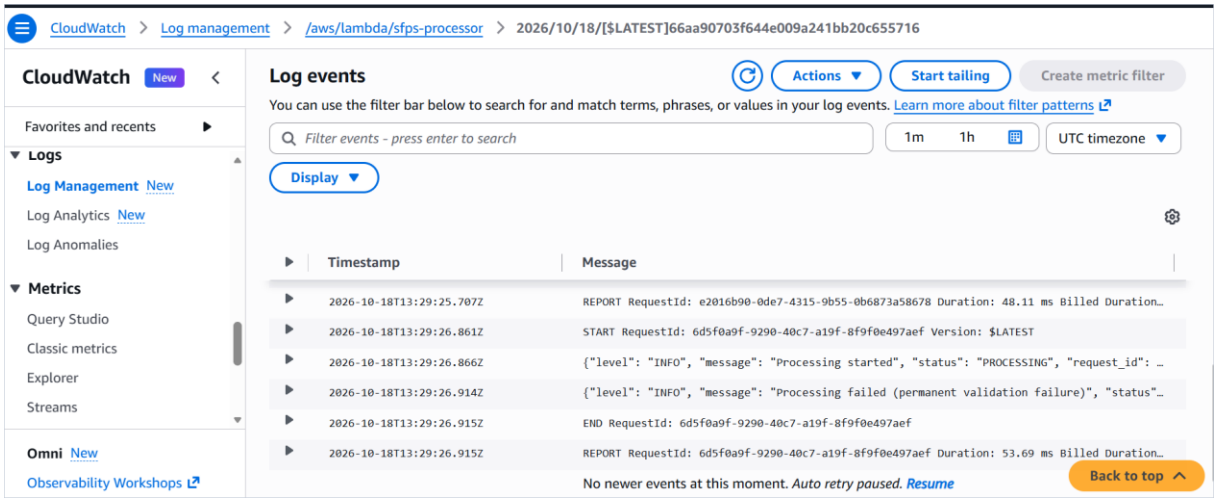


Figure 9.8. *CloudWatch log events for a permanent validation failure, showing platform lines and structured application lines.*

The captured REPORT durations for the failed invocations are 48.11, 53.69, 104.64, and 339.63 ms across the two log views. These are observations, not benchmarks. They show that a validation failure can complete quickly and that invocation overhead varies. They do not describe successful image processing, and they are not averages.

9.10 SQS and retry verification

An infrastructure failure was intentionally triggered by changing the function's TABLE_NAME to a table the execution role cannot access. The role is scoped to sfps-processing, so the targeted call fails with an authorization error rather than a resource-not-found error. The handler logs sanitized diagnostics and raises the fixed InfrastructureError. Lambda retries the function error, and after the retries are exhausted the event is delivered to sfps-failed-events.

The captured queue page shows one message available and one received message with receive count 1. The captured message body has condition RetriesExhausted, approximateInvokeCount 3, the original event under requestPayload for incoming/images.jpg, and a responsePayload containing the fixed message and a stack trace that stops at the handler boundary. This confirms the destination behavior for that injected failure. It does not prove that every possible failure mode produces the same envelope, and it does not prove that the retry timing matches a particular schedule.

The same file name, images.jpg, also appears as a COMPLETED record in the DynamoDB scan. The evidence does not establish whether that completion came from a replay, a new upload, or a retry after the configuration was restored. The document therefore describes the two pieces of evidence separately and does not assert an order that the captures do not show.

9.11 Deployment configuration verification

The console captures verify the function runtime, architecture, memory, timeout, environment variables, trigger prefix and event type, SQS destination, log group existence, IAM role page, and the DynamoDB table browser. The captures do not verify the bucket policy text, the lifecycle rule, the public-access settings, the encryption setting, the exact IAM policy document, the log retention value, the async retry count, or the queue retention and visibility settings. Those are documented configuration items with no direct visual proof in the supplied evidence. The deployment guide contains the commands and console paths to check them.

9.12 Resource-boundary testing

The repository includes boundary tests for the size limit, the pixel budget, and the text counters. The handler accepts an object exactly at the 25 MiB limit and rejects one byte over. The image validator accepts a 4000 by 4000 PNG and rejects a 4001 by 4000 PNG. An animated PNG budget test counts every frame canvas. A 12,000 by 12,000 PNG is rejected before decode with IMAGE_DIMENSIONS_EXCEEDED. A 200,000-line text fixture is counted with less than 1 MiB of extra allocation. These tests ran locally on Windows Python 3.13.14.

The local boundary tests do not certify the deployed Lambda memory or timeout. The build state explicitly says that host measurements are not Linux arm64 Lambda metrics. The deployed oversized-image case is observed as a FAILED record, but the 25 MiB text case and the exact pixel-boundary image case were not measured on the deployed function in the supplied evidence. That remains an outstanding acceptance item.

9.13 Performance observations

Performance evidence is limited. The most specific observation is a single successful PNG invocation recorded by the owner at 89 ms of application processing, 150.13 ms of Lambda execution, and 117 MB of peak memory against a 512 MB configuration. The 89 ms figure is the handler's processing_duration; the 150.13 ms figure is the Lambda REPORT duration. They measure different intervals and should not be added or averaged together.

The failed-invocation REPORT durations visible in CloudWatch range from 48.11 ms to 339.63 ms. The variation is consistent with different validation paths, cold versus warm execution environments, and normal service overhead. The evidence does not include a controlled cold start study, a throughput test, a concurrency test, or a 25 MiB boundary measurement. The case study does not present these numbers as a benchmark.

9.14 Deployed test matrix

Table 9.3. *Deployed functional and validation test matrix.*

Test ID	Test scenario	Input	Expected result	Actual result	Status	Evidence
DEP-01	JPEG success	sample.jpg, 141,143 bytes	COMPLETED with JPEG metadata and SHA-256.	COMPLETED record; no error map.	Pass	Figure 9.4; scan row.
DEP-02	PNG success	sample.png, smaller version	COMPLETED with PNG metadata and SHA-256.	COMPLETED record; no error map.	Pass	Figure 9.4; scan row.
DEP-03	UTF-8 text success	sample.txt, 25 bytes	COMPLETED with character, word, and line counts.	COMPLETED record; no error map.	Pass	Figure 9.6; scan row.
DEP-04	Unsupported PDF	sample.pdf, 1.6 MB	FAILED with UNSUPPORTED_EXTENSION; no download.	FAILED; error map present. Exact code truncated in table view.	Pass, exact code owner-reported	Figure 9.4; scan row.
DEP-05	Oversized image dimensions	sample.png, larger version, 411,069 bytes	FAILED with IMAGE_DIMENSIONS_EXCEEDED .	FAILED; error map present. Exact code truncated.	Pass, exact code owner-reported	Figure 9.4; scan row.
DEP-06	Zero-byte folder marker	incoming/ marker	FAILED with ZERO_BYTE.	FAILED; 0 bytes; empty file name; error map present.	Pass	Figures 9.5 and 9.6; scan rows.
DEP-07	Landscape JPEG	01-landscape.jpg, 9,014 bytes	COMPLETED.	COMPLETED; no error map.	Pass	Figures 9.3 and 9.5.
DEP-08	Portrait JPEG	02-portrait.jpeg, 8,568 bytes	COMPLETED.	COMPLETED; no error map.	Pass	Figures 9.3 and 9.6.
DEP-09	Square PNG	03-square.png, 2,835 bytes	COMPLETED.	COMPLETED; no error map.	Pass	Figures 9.3 and 9.6.
DEP-10	Multiline text	04-multiline.txt, 47 bytes	COMPLETED with line counts.	COMPLETED; no error map.	Pass	Figures 9.3 and 9.4.
DEP-11	Unicode text	05-unicode.txt, 72 bytes	COMPLETED with Unicode counts.	COMPLETED; no error map.	Pass	Figures 9.3 and 9.5.
DEP-12	Nested image	06-nested-image.jpg, 12,651 bytes	COMPLETED in a nested prefix.	COMPLETED; no error map.	Pass	Figures 9.3 and 9.7.
DEP-13	Empty text	07-empty.txt, 0 bytes	FAILED with ZERO_BYTE.	FAILED; error map present.	Pass	Figure 9.6; scan row.
DEP-14	Unsupported CSV	08-unsupported.csv, 18 bytes	FAILED with UNSUPPORTED_EXTENSION.	FAILED; error map present.	Pass	Figures 9.2 and 9.6.
DEP-15	Invalid UTF-8	09-invalid-utf8.txt, 11 bytes	FAILED with INVALID_ENCODING.	FAILED; error map present.	Pass	Figures 9.2 and 9.7.
DEP-16	Fake JPEG	10-fake-jpeg.jpg, 39 bytes	FAILED with TYPE_MISMATCH.	FAILED; error map present.	Pass	Figures 9.2 and 9.7.
DEP-17	Corrupt PNG	11-corrupt-png.png, 177 bytes	FAILED with CORRUPT_IMAGE.	FAILED; error map present.	Pass	Figure 9.5; scan row.
DEP-18	Over pixel limit	12-over-pixel-limit.png, 61,158 bytes	FAILED with IMAGE_DIMENSIONS_EXCEEDED .	FAILED; error map present.	Pass	Figures 9.2 and 9.6.

9.15 Automated and boundary test matrix

Table 9.4. *Automated, regression, and boundary test matrix.*

Test ID	Test scenario	Input	Expected result	Actual result	Status	Evidence
AUT-01	Duplicate delivery of a completed version	Same event twice in moto.	Second invocation skips; record unchanged.	Record byte-for-byte unchanged; duplicate logged.	Pass	test_handler.py.
AUT-02	Re-attempt of a PROCESSING record	Pre-seeded PROCESSING item.	Processing continues; created_at preserved.	COMPLETED; original created_at intact.	Pass	test_handler.py.
AUT-03	Terminal conditional race	Racing completion injected during hashing.	Terminal update does not overwrite the racer.	Racer's processed_at preserved.	Pass	test_handler.py.
AUT-04	Missing version ID	Record without versionId.	MALFORMED_EVENT; no record.	Raised; scan empty.	Pass	test_handler.py.
AUT-05	Invalid identity types	Numeric, bool, list, object, null.	MALFORMED_EVENT; no record per identity.	Raised; no records written.	Pass	test_audit_regressions.py.
AUT-06	Unsupported extension before download	Spy on get_object.	FAILED; get_object not called.	No download; FAILED with expected code.	Pass	test_handler.py.
AUT-07	HeadObject service error	AccessDenied from HeadObject.	FAILED with INFRASTRUCTURE; InfrastructureError raised.	Observed.	Pass	test_handler.py.
AUT-08	Unexpected decoder error	MemoryError or RuntimeError from Pillow.	Infrastructure path; no raw message persisted.	Observed; marker absent.	Pass	test_audit_regressions.py.
AUT-09	PNG checksum damage	IDAT byte flipped.	FAILED with CORRUPT_IMAGE.	Observed.	Pass	test_audit_regressions.py.
AUT-10	Error serialization cap	5,000-character message and escaped text.	Serialized error at most 1,024 characters; code and type intact.	Observed.	Pass	test_handler.py.
AUT-11	Logger boundary	Root handler installed.	No application record reaches the root handler.	Observed.	Pass	test_audit_regressions.py.
AUT-12	25 MiB size boundary	Exactly 25 MiB and 25 MiB + 1 byte text.	Exact limit accepted; over limit rejected.	Observed locally.	Pass locally; not deployed	test_handler.py.
AUT-13	Pixel boundary	4000x4000 and 4001x4000 PNG.	Exact budget accepted; over budget rejected.	Observed locally.	Pass locally; not deployed	test_audit_regressions.py.
AUT-14	All-frame budget	Animated PNG with two canvases.	Budget counts every frame.	Observed.	Pass	test_audit_regressions.py.

Test ID	Test scenario	Input	Expected result	Actual result	Status	Evidence
AUT-15	Text memory bound	200,000-line text fixture.	Correct counts; extra allocation below 1 MiB.	Observed.	Pass	test_audit_regressions.py.

9.16 Test status and remaining gaps

The automated suite is comprehensive for the handler and pure modules. The deployed evidence is strong for the resource topology, the positive and negative acceptance files, the DynamoDB records, the CloudWatch log format, and one injected SQS failure. The evidence is weaker for the exact IAM policy attachment, the bucket configuration pages, the log retention value, the async retry count, and the 25 MiB memory and timeout boundary. The case study marks those as implemented but not independently verified rather than filling the gaps with assumptions.

Three tests remain outstanding for a full acceptance close-out. First, measure a 25 MiB text file and a pixel-boundary image on the deployed function and record REPORT memory and duration. Second, capture the bucket Properties and Permissions pages plus the attached IAM policy JSON. Third, capture the log group retention setting and the asynchronous invocation retry count. Those captures would convert several documented items into visually verified evidence.

Chapter 10. Results and Performance Analysis

10.1 Evidence summary

The results come from three sources: the repository test suite, the deployed console captures, and the captured SQS envelope. The test suite establishes the behavior of the code under controlled inputs. The console captures establish that the resources exist and that the acceptance files produced records. The envelope establishes that one infrastructure failure exhausted retries and reached the SQS destination. The results below separate those sources instead of blending them into a single success claim.

10.2 Processing results

The deployed acceptance run produced COMPLETED records for the initial JPEG, PNG, and text files, plus images.jpg. The additional positive suite produced COMPLETED records for all six files: 01-landscape.jpg, 02-portrait.jpeg, 03-square.png, 04-multiline.txt, 05-unicode.txt, and 06-nested-image.jpg. The DynamoDB scan shows no error map on those rows. The record names and sizes match the S3 listings where both are visible. This is the strongest end-to-end evidence in the project: an upload became an event, the event became a function invocation, and the invocation became a durable record.

The table also contains a COMPLETED record for sample.png with a smaller size and a FAILED record for sample.png with a larger size. The two records are different S3 versions of the same key. The completed version passed validation; the larger version exceeded the pixel budget. This is the versioned-identity model working as designed. The same bytes did not receive two different outcomes.

10.3 Validation results

The deployed negative suite produced FAILED records for all six files. The initial negative set produced FAILED records for sample.pdf and the larger sample.png version. The seven zero-byte folder markers also produced FAILED records. The FAILED records carry an error map; the console table truncates the map, so the screenshots confirm failure and the presence of an error rather than the exact code. The owner-reported results and the validation order support the expected codes: ZERO_BYTE, UNSUPPORTED_EXTENSION, INVALID_ENCODING, TYPE_MISMATCH, CORRUPT_IMAGE, and IMAGE_DIMENSIONS_EXCEEDED.

The failure classification behaved as intended at the batch level. The permanent failures did not loop. The initial acceptance set returned successfully even though it contained a PDF and zero-byte markers. The SQS queue did not receive those permanent failures. The one captured SQS message came from an intentionally injected infrastructure failure, which is the correct channel for that class of error.

10.4 State persistence and metadata

The DynamoDB scan shows the processing_id, bucket, created_at, file_name, file_size, and error columns. It does not show sha256, metadata, processed_at, or the full error map in the captured views. The automated tests verify the full record shape for JPEG, PNG, and text, including the exact metadata dictionaries and SHA-256 values. The deployed screenshots verify that the records were written and that the statuses are correct. The combination supports the schema and the persistence behavior without pretending that every attribute is visible in a console table.

The automated text tests assert that a trailing newline does not add a line, that whitespace-only text has zero words, that Unicode counts code points rather than bytes, and that CRLF is handled. The image tests assert exact width, height, and format for JPEG and PNG. The handler tests assert that these metadata dictionaries are stored under the metadata attribute and that the checksum matches hashlib.sha256 over the same bytes. The deployed records are consistent with those contracts.

10.5 Nested-path support

The positive suite includes a nested/ folder and a 06-nested-image.jpg record. The record completed with no error map. The handler unit tests also cover incoming/2026/photo.jpg and incoming/users/demo/file.txt. The S3 trigger's incoming/ prefix applies to all nested keys. The combination verifies that the deployed pipeline is not limited to a flat upload folder.

10.6 Versioning behavior

The two sample.png records demonstrate the versioning behavior in the deployed table. The smaller version completed; the larger version failed. The record identity includes the version ID, so the two versions do not collide. The SQS envelope also carries a versionId for images.jpg, which confirms that the event payload includes the version field the handler requires. The lifecycle rule that expires noncurrent versions after 30 days is documented but not visible in the captures.

10.7 CloudWatch observations

The log group contains seven recent streams. The captured log events show the platform START, END, and REPORT lines and application JSON lines. The application lines carry the status field and a message. The JSON values are truncated by the console width, so the full object_key, version_id, and error fields are not legible in the capture. The log format is verified; the complete field values are not. The application logger's non-propagation behavior and its sanitized exception rendering are verified by tests rather than by the console view.

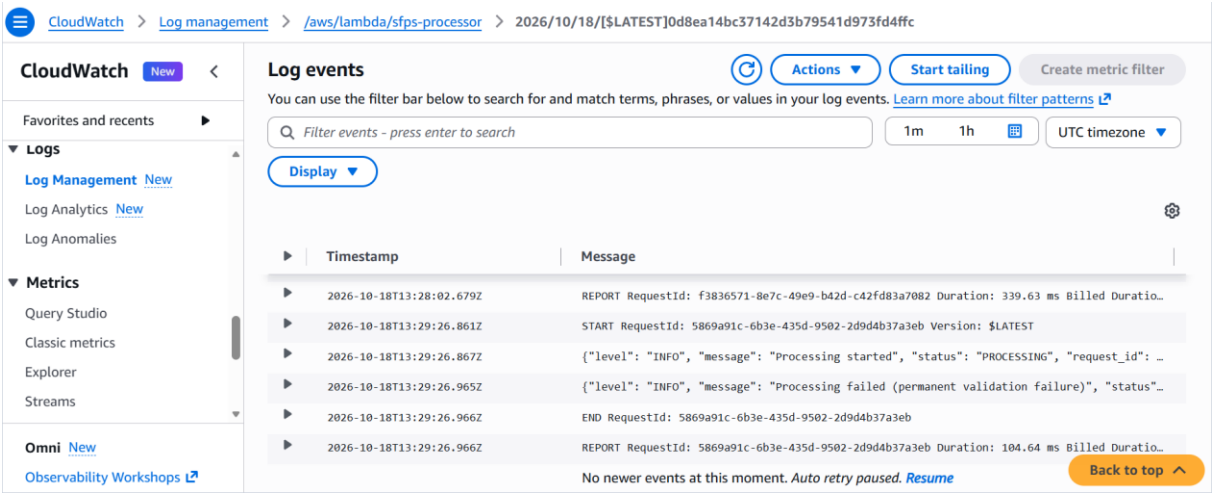


Figure 10.1. *CloudWatch log events showing two failed invocations and their REPORT durations.*

The REPORT durations in the captured failed invocations are 48.11, 53.69, 104.64, and 339.63 ms. These numbers are useful for showing that the function returns quickly on a validation failure. They are not a throughput measurement, a cold-start measurement, or a successful image-processing benchmark. The variation may come from execution environment reuse, validation path differences, or normal service overhead. The evidence does not isolate those factors.

10.8 Performance observations

The single recorded successful PNG invocation is the most specific performance observation in the project. The owner recorded 89 ms of application processing, 150.13 ms of Lambda execution, and 117 MB of peak memory against a 512 MB configuration. The application duration is measured by the handler around the processing pipeline. The Lambda duration is measured by the platform around the invocation. The memory figure is the platform's peak for that invocation. One invocation does not establish an average, a percentile, or a capacity plan.

Table 10.1. *Observed performance values and their scope.*

Metric	Observed value	Scope	Source
Application processing duration	89 ms	One successful PNG invocation.	Owner-recorded observation; not visible in the supplied screenshots.
Lambda execution duration	150.13 ms	Same invocation.	Owner-recorded observation; not visible in the supplied screenshots.
Peak memory	117 MB	Same invocation.	Owner-recorded observation; not visible in the supplied screenshots.
Configured memory	512 MB	Deployed function.	Figure 6.5.
Configured timeout	30 seconds	Deployed function.	Figure 6.5.
Failed validation REPORT duration	48.11 ms, 53.69 ms, 104.64 ms, 339.63 ms	Four captured failed invocations; not an average.	Figures 9.8 and 10.1.
Local 25 MiB text processing	4.68 seconds; 69.4 MiB peak working set	Windows Python 3.13.14 host measurement, not Lambda.	BUILD_STATE 2026-10-16.
Local 4000x4000 PNG	Accepted; 82.7 MiB process peak	Windows host measurement, not Lambda.	BUILD_STATE 2026-10-16.
Local 12000x12000 PNG	Rejected before decode; process peak remained 82.7 MiB	Windows host measurement, not Lambda.	BUILD_STATE 2026-10-16.

The local measurements are included because they explain why the resource bounds were added. The 25 MiB text case previously used 548.8 MiB of working set under the list-based implementation; the iterator-based implementation reduced it to 69.4 MiB. That is a meaningful engineering result, but it was measured on Windows, not on the deployed Linux arm64 function. The case study does not convert it into a Lambda guarantee.

10.9 Reliability behavior

The captured SQS envelope is the strongest reliability evidence. It shows that an injected infrastructure failure produced a RetriesExhausted condition, an approximate invoke count of 3, the original event under requestPayload, and a sanitized responsePayload. That is the expected behavior for a function error that exhausts the asynchronous retry budget. The queue page shows one message available and one received message with receive count 1. The message was not deleted in the capture, which is consistent with the absence of a consumer.

The automated tests provide complementary reliability evidence: duplicate delivery is skipped, PROCESSING is re-attempted, a terminal race does not overwrite a completed record, a malformed event writes nothing, a HeadObject error becomes a FAILED infrastructure record, and a secondary failure is logged without replacing the original diagnostic. These tests use moto, so they do not prove the deployed IAM or network behavior. They do prove the handler's state machine and error classification.

10.10 Claims not made

The project does not claim that the deployed function meets a particular latency service level, that it sustains a particular request rate, that it is cheaper than a server-based worker at the project's actual volume, or that it has been tested against malicious files beyond the listed fixtures. It does not claim that the validation is exhaustive for every malformed image or Unicode edge case. It does not claim that the error codes visible in the console are complete for every record, because the table view truncates them. It does not claim that the system is production-ready without the enhancements in Chapter 13.

Chapter 11. Challenges and Solutions

The project produced several engineering difficulties that were not obvious from the requirements alone. Each one changed the implementation or the documentation. The sections below describe the cause, the investigation, the resolution, and the lesson. Only issues that appear in the repository history, the build state, or the deployment evidence are included.

11.1 Packaging Linux ARM64 dependencies from Windows

The build host is Windows and the Lambda target is Linux arm64. A normal pip install on the host resolves Windows wheels, including Windows .pyd extension modules for Pillow. Those modules cannot load on Lambda. The investigation compared the target runtime (Python 3.13 on Amazon Linux 2023, arm64) with the host platform and identified the need for a cross-platform wheel download. The resolution uses pip's --python-version, --implementation, --abi, --platform, and --only-binary options to fetch manylinux_2_28_aarch64 cp313 wheels. The lesson is that a deployment package is a target artifact, not a copy of the developer environment.

11.2 Lambda ZIP archive layout

Lambda imports the handler by module path. A ZIP that contains a top-level src/ folder puts handler.py at src/handler.py, which does not match handler.lambda_handler. A ZIP built with PowerShell 5.1 Compress-Archive can also contain backslash entry names that Lambda cannot unpack. The investigation reproduced both failure modes in the packaging checks. The resolution copies the contents of src/ into a flat build directory, zips the contents with Python's zipfile, replaces os.sep with a forward slash, and asserts that handler.py is at the root, no entry contains a backslash, and no entry starts with ./. The lesson is that archive layout is part of the application contract.

11.3 S3 versioning and version-pinned reads

Versioning is required for the processing identity, but it also changes how reads must be performed. An event refers to a specific version. If the handler calls GetObject without a version ID, it may read a newer object than the event described. The investigation tied the processing identity to the version ID and added VersionId to both HeadObject and GetObject. A missing or null version ID is now a malformed event. The lesson is that an event's identity must travel with every read that depends on it.

11.4 IAM permission scope

An early deployment can fail with AccessDenied if the policy ARNs still contain placeholders or if the S3 resource is scoped to the bucket ARN without the incoming/ prefix. The investigation compared each policy statement with the actual resource ARN and the operations the handler performs. The resolution uses a prefix-scoped S3 resource, a table-scoped DynamoDB resource, a queue ARN for SendMessage, and a log-group ARN with the :* suffix. The lesson is that a policy is only least-privilege if its resource scope matches the code's actual access pattern.

11.5 Zero-byte folder markers

Creating a folder in the S3 console creates a zero-byte object whose key ends with a slash. Those objects match the incoming/ prefix and invoke the function. The first reaction is to treat them as noise. The investigation showed that they are valid S3 objects and that the pipeline should record them rather than ignore them. The resolution keeps the zero-byte check before the extension check, so a folder marker becomes FAILED with ZERO_BYTE. The result is seven documented folder-marker records in the scan. The lesson is that console conveniences create real objects, and the pipeline's behavior should be predictable for them.

11.6 Handling unsupported extensions

An unsupported file must not disappear silently. A suffix filter on the S3 notification would prevent a PDF from reaching Lambda, but it would also remove any record of the rejection. The resolution uses no suffix filter and checks the extension allowlist after the size check and before the content download. The PDF becomes a FAILED record with UNSUPPORTED_EXTENSION and does not consume bandwidth for its 1.6 MB body. The lesson is that observability sometimes requires accepting an event you intend to reject.

11.7 Detecting fake or corrupt images

An extension check is not enough. A file named .jpg can contain PNG bytes, text, or a truncated JPEG. The investigation added a signature check, a Pillow verify pass, a reopen, and a full decode of every frame. A signature mismatch becomes TYPE_MISMATCH; a matching signature that cannot be verified or decoded becomes CORRUPT_IMAGE. The regression suite includes a PNG with a damaged IDAT checksum and a truncated JPEG. The lesson is that image validation needs both a cheap signature check and a real decoder pass.

11.8 Oversized image validation

A small file can decode into a very large pixel buffer. The investigation found a 420 KB PNG that exceeded 512 MiB when decoded. The resolution adds a total-pixel budget of 16,000,000 across all frame canvases, checked before allocation. A 12,000 by 12,000 PNG is rejected before decode. Animated PNGs are covered because the budget multiplies by the frame count. The lesson is that upload size is not a proxy for decode cost.

11.9 UTF-8 decoding failures

Text files can contain invalid byte sequences. A permissive decode would replace them with replacement characters and produce misleading counts. The resolution uses strict UTF-8 decode, maps failures to INVALID_ENCODING, and strips exactly one leading BOM after a successful decode. A BOM-only file becomes an empty string and is valid at the text layer; a zero-byte object is rejected earlier. The lesson is that text statistics are only meaningful after a strict decode.

11.10 Asynchronous retry troubleshooting

An asynchronous invocation failure is not visible in the same way as a synchronous request. The function's exception is logged, but the retry and destination behavior is controlled by Lambda. The investigation used the SQS destination envelope to confirm the condition and invoke count. The captured message shows RetriesExhausted and approximateInvokeCount 3. The resolution is a documented failure-injection procedure: point TABLE_NAME at a table the role cannot access, upload a supported file, wait for retries, inspect the queue, then restore the variable. The lesson is that retry behavior must be observed at the destination, not inferred from a function log alone.

11.11 Distinguishing validation failures from infrastructure exceptions

The first failure model treated every exception as retryable. That would turn a corrupt image into an endless retry loop. The investigation separated ValidationError from unexpected exceptions, added a guarded FAILED write, and moved the fixed boundary error to the top of the handler. The resolution makes a validation failure return successfully and an infrastructure failure raise. The lesson is that retry semantics are a property of the error class, not a global policy.

11.12 Evidence drift and stale build artifacts

The workspace contains an older build/ directory with botocore 1.43.113, while the verified function.zip contains 1.43.112. A casual review could describe the wrong package. The investigation compared the archive metadata with requirements.txt and treated function.zip as the release artifact. The resolution documents the stale directory and cites the archive, not the working directory. The lesson is that evidence has a provenance, and the newest-looking folder is not always the released artifact.

Chapter 12. Cost and Operational Considerations

12.1 Pay-per-use model

Every service in the pipeline is pay-per-use. There is no idle server, no provisioned database capacity, and no reserved concurrency. The cost is driven by the number and size of uploaded objects, the duration and memory of each Lambda invocation, the number of DynamoDB reads and writes, the number of SQS messages, and the volume of logs retained. The project did not collect billing data, so this chapter describes the cost model qualitatively. It does not present dollar figures.

The acceptance workload was tiny: a few dozen uploads, a handful of image decodes, and a small number of log lines. Any real bill for that workload would be dominated by rounding, free-tier allowances, or the minimum billable units rather than by the architecture. A numeric cost claim without a billing export would be misleading.

12.2 Amazon S3

S3 charges for storage, requests, and data transfer. The project stores small source files and keeps current versions indefinitely plus noncurrent versions for 30 days. The main cost drivers are the number and size of stored versions, the number of PUT and GET requests, and any data transfer out of the region. The function's reads are version-pinned GET and HEAD requests. The bucket has no public website endpoint, so there is no public transfer path.

The lifecycle rule is a cost control as well as a retention control. Expiring noncurrent versions after 30 days prevents repeated uploads from accumulating storage indefinitely. It does not delete current versions, so the latest object for each key remains available.

12.3 AWS Lambda

Lambda charges for the number of requests and for compute duration multiplied by the configured memory. The function runs at 512 MB with a 30-second timeout. The observed failed validation invocations completed in tens to hundreds of milliseconds. The one recorded successful PNG invocation used 150.13 ms of Lambda duration and 117 MB of peak memory. The memory setting is the billing lever: a higher setting can finish sooner but costs more per millisecond; a lower setting may take longer or fail on larger images.

Asynchronous retries can multiply the invocation count for a failing event. A persistent infrastructure failure can consume the initial attempt plus two function-error retries, and throttling or system errors can add more retries up to the maximum event age. The SQS destination itself is not a Lambda charge, but the extra invocations are. This is one reason the failure classification matters for cost as well as correctness.

12.4 Amazon DynamoDB

The table uses on-demand capacity, so the project pays per read and write request unit rather than for provisioned capacity. Each record involves a conditional PutItem, possibly a GetItem after a claim conflict, and a conditional UpdateItem. The scan used for verification is a table scan and consumes read capacity proportional to the number and size of items. A production workload would use targeted queries or a secondary index instead of repeated scans.

The table has no TTL, so records accumulate. That is appropriate for an operational ledger, but it is a cost consideration over time. A future retention policy could archive or expire old records. The project does not enable point-in-time recovery, which avoids the PITR storage cost but also removes a recovery option.

12.5 Amazon SQS

SQS charges per request, with a free tier for standard queues. The queue receives messages only when an asynchronous invocation exhausts its retries, so the request volume is expected to be low in normal operation. The 14-day retention keeps a failed message available for investigation without an immediate consumer. A production design would add a consumer or an alert, which would add its own request and compute cost.

12.6 Amazon CloudWatch Logs

CloudWatch Logs charges for ingestion and storage. The application writes one JSON start line and one terminal line per record, plus diagnostic lines for failures. The Lambda platform adds START, END, and REPORT lines. The 30-day retention bounds storage growth. A high-volume workload would want a log-level policy, structured filtering, and possibly a shorter retention for verbose diagnostics. The project keeps INFO logging and 30-day retention as a simple operational baseline.

12.7 Cost drivers

Table 12.1. *Cost drivers and qualitative controls.*

Service	Main cost drivers	Project controls	Operational note
Amazon S3	Stored bytes, PUT/GET/HEAD requests, noncurrent versions, data transfer.	Private bucket; 30-day noncurrent expiry; no public endpoint.	Repeated uploads create versions; the lifecycle rule bounds old-version storage.
AWS Lambda	Request count, duration, configured memory, retries.	512 MB; 30-second timeout; permanent failures do not retry.	A persistent infrastructure failure can multiply invocations through retries.
Amazon DynamoDB	Read and write request units; table scans; storage.	On-demand capacity; one item per version; targeted item reads in the handler.	Verification scans are not representative of steady-state traffic.
Amazon SQS	Send, receive, and delete requests; retention.	Messages only for exhausted failures; 14-day retention; no consumer in V1.	A consumer or replay tool would add requests.
Amazon CloudWatch Logs	Ingestion volume, storage, retention.	Two application lines per record; 30-day retention.	Verbose logging and long retention increase cost.

12.8 No billing evidence

The project did not export Cost Explorer data, a billing report, or a budget. The case study therefore does not state an actual monthly cost, a cost per file, or a comparison with a server-based alternative. Those claims would require a measured workload and a billing period. The qualitative model is enough to explain what would drive the bill and which design choices bound it.

12.9 Operational considerations

The operational burden is mostly configuration and observation. The manual deployment requires careful ARN replacement, a pre-created log group, and a versioning propagation wait. Once deployed, the routine checks are the DynamoDB table for FAILED records, CloudWatch for diagnostic lines, and SQS for unexpected messages. There is no server to patch, but there are managed-service quotas, runtime deprecation dates, and dependency updates to track. A production operation would add alarms, a runbook, and a replay procedure before it could claim a meaningful service level.

Chapter 13. Limitations and Future Enhancements

13.1 Current limitations

V1 is a manual deployment. There is no infrastructure as code, no CI/CD pipeline, and no automated environment promotion. The function handles four file extensions and does not process PDFs, video, archives, or office documents. It does not resize images, scan for malware, deduplicate content by checksum, or enrich metadata beyond dimensions and text counts. It does not use a DynamoDB transaction, a dead-letter queue for SQS, or a consumer that replays failed events.

The observability is basic. There are no CloudWatch alarms, dashboards, or metric filters. A message in the failure queue is visible only if an operator looks. The table has no point-in-time recovery and no secondary indexes. The function has no reserved concurrency. The deployment package is built and uploaded by hand. The 25 MiB memory and timeout boundary has not been measured on the deployed function. The IAM policy and bucket configuration are documented but not all of their values are visually verified in the captures.

13.2 Future enhancements

The most valuable next step is infrastructure as code. A SAM or Terraform definition would make the bucket, table, queue, role, function, trigger, destination, and log group reproducible and reviewable. It would also remove the placeholder-replacement errors that manual deployment invites. A CI/CD pipeline could run the test suite, build the arm64 package, and deploy to a staging account after approval. The current repository is already structured for that: the tests are offline, the package build is scripted, and the configuration is explicit.

The second step is operational monitoring. CloudWatch alarms on Lambda errors, throttles, duration, and SQS visible messages would turn a silent failure into a notification. A metric filter on the application FAILED status could separate validation failures from infrastructure failures. A dashboard could show uploads, completions, rejections, and queue depth. A replay tool could read a failure envelope, verify that the original event is well formed, and invoke the function asynchronously after the cause is fixed.

The third step is broader validation and load testing. The boundary tests should run on the deployed function with a 25 MiB text file and a pixel-boundary image, recording REPORT memory and duration. A concurrency test could measure how the function behaves when many objects arrive at once. A fuzzing pass could feed malformed image headers, truncated text, and unusual Unicode. A cost test could compare on-demand DynamoDB with a provisioned or adaptive baseline for a realistic workload.

Feature enhancements could add PDF text extraction, image thumbnails, malware scanning, or content deduplication. Each adds a new validation surface, a new dependency, and a new failure mode. They should follow the same pattern as the current types: a signature or structure check, a bounded decode, a fixed metadata schema, and a clear error taxonomy. Adding formats without those controls would weaken the design.

13.3 Prioritized roadmap

Table 13.1. *Limitations and prioritized enhancements.*

Area	Current limitation	Enhancement	Priority
Provisioning	Manual console deployment.	SAM or Terraform definition with parameterized region and account.	High
Delivery	Manual build and upload.	CI pipeline that runs tests, builds the arm64 package, and deploys after approval.	High
Monitoring	No alarms or dashboard.	CloudWatch alarms for Lambda errors, throttles, duration, and SQS depth; FAILED-status metric filter.	High
Failure handling	No replay tool or SQS consumer.	Replay runbook and tool for requestPayload; optional consumer for triage.	High
Boundary acceptance	25 MiB memory and timeout not measured on Lambda.	Deployed boundary test with REPORT capture.	High
Evidence	IAM policy and bucket settings not visually verified.	Capture policy JSON, bucket Properties, and log retention.	Medium
Data recovery	PITR disabled; no archive.	Enable PITR or export records to a durable archive.	Medium
Formats	Four extensions only.	Add PDF or office formats with bounded parsing and explicit error codes.	Medium
Security	No malware scanning; SSE-S3 only.	Malware scanning step; optional KMS customer-managed key.	Medium
Load	No throughput or concurrency test.	Controlled load test and reserved concurrency decision.	Low
Cost	No billing data or budget.	Cost allocation tags, budget alarm, and monthly review.	Low
Documentation	No runbook for on-call.	Operational runbook with triage, replay, and rollback steps.	Low

13.4 What is not planned

The project does not plan to add a web frontend, API Gateway, Cognito, CloudFront, Step Functions, SNS notifications, or a container image for V1. Those services could be useful in a different product, but they would distract from the processing pipeline and the evidence. The same applies to multi-region replication and a full observability platform. The roadmap prioritizes the controls that make the existing pipeline reproducible and observable.

Chapter 14. Conclusion and Lessons Learned

14.1 Demonstrated outcomes

SFPS demonstrates a complete event-driven file-processing pipeline on AWS. An upload to a private, versioned S3 bucket invokes a Python 3.13 Lambda function. The function validates the file, extracts metadata, computes a SHA-256 checksum, and writes one deterministic DynamoDB record per object version. Structured JSON logs go to CloudWatch. Permanent validation failures become FAILED records and do not retry. Infrastructure failures are retried and, when the retries are exhausted, delivered to an SQS queue. The deployed evidence includes a 25-item DynamoDB scan with 10 COMPLETED and 15 FAILED records, a captured SQS envelope with RetriesExhausted and approximateInvokeCount 3, and a single successful PNG invocation at 89 ms of application time, 150.13 ms of Lambda time, and 117 MB of peak memory.

The project also demonstrates the engineering discipline around the pipeline. The repository contains 135 automated tests, a clean Ruff run, a dependency lock, a verified Linux arm64 deployment archive, a product requirements document, an architecture decision log, a build state, a console deployment guide, and this evidence-backed case study. The claims are separated by evidence layer, and the gaps are named rather than hidden.

14.2 Design decisions that mattered

Four decisions carried most of the design. The deterministic processing identity made idempotency possible without a separate deduplication service. The version-pinned read made the record describe the bytes that were actually processed. The conditional write protocol made concurrent and duplicate invocations safe. The failure classification kept expected rejections out of the retry loop while preserving a path for genuine operational faults. Each decision is small in code and large in consequence.

A fifth decision was to keep the pure logic separate from AWS orchestration. The validation, checksum, and processor modules do not import boto3. That separation made the unit tests fast and focused, and it made the handler tests about orchestration rather than decoding. The project structure is not elaborate, but the seam is in the right place.

14.3 Reliability lessons

The project reinforced a few lessons that apply beyond AWS. At-least-once delivery is the default assumption, not an edge case. A retry policy must be tied to the error class. A record should identify the exact input it processed, including its version, rather than its file name alone. A build artifact needs provenance, because a working directory can contain stale dependencies. A console screenshot proves only what it shows; the rest is documentation, and the difference should be stated.

The evidence gaps are part of the result. The 25 MiB memory and timeout boundary, the attached IAM policy, the bucket configuration pages, and the log retention value are not fully verified in the supplied captures. Naming those gaps is more useful than filling them with confident prose. A reviewer can run the remaining checks with the deployment guide, and the case study will still be accurate.

14.4 Final assessment

SFPS is a credible V1 serverless file-processing system. It handles the supported formats, records every object version, separates expected rejections from operational faults, and produces enough evidence to audit the main claims. It is a well-scoped engineering project rather than a production platform, and it shows how the pieces fit together and where the next layer of work would go.

The most useful outcome is the habit of asking what each claim rests on, more than the list of AWS services. The repository tests, the deployed captures, the SQS envelope, and the owner-recorded metrics each answer a different question. Keeping those answers separate is what makes the project defensible.

Appendix A. Configuration Reference

Table A.1. *Configuration reference for the deployed V1 environment.*

Setting	Value	Notes
Region	ap-south-1	All resources use the same region.
S3 bucket	sfps-incoming-<account-id>	Globally unique; lowercase.
S3 prefix	incoming/	Exact, lowercase, trailing slash.
S3 versioning	Enabled	Required for version IDs in events.
S3 public access	Block all public access	All four settings enabled.
S3 encryption	SSE-S3	S3 Bucket Keys apply to SSE-KMS, not SSE-S3.
S3 TLS policy	DenyNonTLSRequests	Deny when aws:SecureTransport is false.
S3 lifecycle	expire-noncurrent-30d	Noncurrent versions deleted after 30 days.
DynamoDB table	sfps-processing	Partition key processing_id (String); no sort key.
DynamoDB capacity	On-demand	No provisioned throughput.
DynamoDB recovery	PITR disabled	No secondary indexes.
SQS queue	sfps-failed-events	Standard; 14-day retention; 30-second visibility; SSE-SQS; no DLQ.
IAM role	sfps-lambda-role (documented); captured role page sfps-lambda-scoped-policy	Trusts lambda.amazonaws.com; inline policy scoped to the project resources.
Lambda function	sfps-processor	Python 3.13; arm64; 512 MB; 30 seconds.
Lambda handler	handler.lambda_handler	handler.py at the ZIP root.
Lambda environment	TABLE_NAME=sfps-processing; MAX_FILE_SIZE_BYTES=26214400; LOG_LEVEL=INFO	Validated at import.
Async retries	MaximumRetryAttempts=2	Maximum event age six hours (default).
Failure destination	SQS sfps-failed-events, On failure	Invocation envelope with requestPayload.
CloudWatch log group	/aws/lambda/sfps-processor	Pre-created; 30-day retention (documented).
S3 trigger	s3:ObjectCreated:*; prefix incoming/; no suffix	Created from the Lambda console.
Upload limit	26,214,400 bytes (25 MiB)	Hard ceiling; MAX_FILE_SIZE_BYTES may lower it.
Image budget	16,000,000 total pixels	width x height x frame count.

Appendix B. Selected JSON and Log Shapes

The SQS envelope below is reproduced from the captured message body for the injected infrastructure failure. Account and network identifiers are redacted. The envelope shows the destination condition, approximate invoke count, the original event under requestPayload, and the sanitized responsePayload. It is not a DynamoDB record and should not be read as one.

Code B.1. *Redacted SQS invocation envelope from the captured failure.*

```
{
  "version": "1.0",
  "timestamp": "2026-10-18T13:10:44.664Z",
  "requestContext": {
    "requestId": "<request-id>",
    "functionArn": "arn:aws:lambda:ap-south-1:<account-id>:function:sfps-processor:$LATEST",
    "condition": "RetriesExhausted",
    "approximateInvokeCount": 3
  },
  "requestPayload": {
    "Records": [{
      "eventSource": "aws:s3",
      "eventName": "ObjectCreated:Put",
      "s3": {
        "bucket": {"name": "sfps-incoming-<account-id>"},
        "object": {
          "key": "incoming/images.jpg",
          "size": 45166,
          "versionId": "38tOf_RG_KCMjbX2xrtDx1vu6ISmXfkA"
        }
      }
    }]
  },
  "responseContext": {"statusCode": 200, "functionError": "Unhandled"},
  "responsePayload": {
    "errorMessage": "File processing failed; see sanitized CloudWatch diagnostic logs",
    "errorType": "InfrastructureError",
    "requestId": "<request-id>"
  }
}
```

The application log schema below is illustrative, not a captured line. The actual captured console lines are truncated by the view width. The schema matches the formatter in handler.py and the tests that assert one-line JSON output.

Code B.2. *Representative application log schema.*

```
{
  "level": "INFO",
  "message": "Processing completed",
  "status": "COMPLETED",
  "request_id": "<lambda-request-id>",
  "event_id": "<sequencer>",
  "processing_id": "<64-hex-character-id>",
  "bucket": "sfps-incoming-<account-id>",
  "object_key": "incoming/example.jpg",
  "version_id": "<version-id>",
  "file_type": "jpg",
  "file_size": 141143,
  "processing_duration": 89
}
```

A FAILED record's error map is smaller. The example below uses the allowlisted message for an unsupported extension. It is a schema example, not a captured DynamoDB item.

Code B.3. *Error map shape for a permanent validation failure.*

```
{
  "error_code": "UNSUPPORTED_EXTENSION",
  "error_type": "ValidationError",
  "message": "File extension is not supported"
}
```

Appendix C. Deployment Commands

Code C.1. *Build, verify, and upload the deployment package.*

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
Remove-Item -Recurse -Force build -ErrorAction SilentlyContinue
python -m pip install --python-version 3.13 --implementation cp --abi cp313 `
  --platform manylinux_2_28_aarch64 --only-binary=:all: --no-compile `
  --target build -r requirements.txt
Get-ChildItem -Path src -Recurse -Directory -Filter __pycache__ |
  Remove-Item -Recurse -Force
Copy-Item -Path src\* -Destination build\ -Recurse -Force
Remove-Item -Force function.zip -ErrorAction SilentlyContinue
python -c "import zipfile, os; z = zipfile.ZipFile('function.zip','w',zipfile.ZIP_DEFLATED); "
  "[z.write(os.path.join(dp,f), os.path.relpath(os.path.join(dp,f),'build').replace(os.sep,'/')) "
  "for dp,dn,fn in os.walk('build') for f in fn]; z.close()"
aws lambda update-function-code --function-name sfps-processor `
  --zip-file fileb://function.zip --region ap-south-1
```

Code C.2. *Optional CLI verification commands.*

```
aws sts get-caller-identity
aws lambda get-policy --function-name sfps-processor --region ap-south-1
aws logs describe-log-groups --log-group-name-prefix /aws/lambda/sfps-processor `
  --region ap-south-1
aws sqs get-queue-attributes --queue-url <queue-url> `
  --attribute-names All --region ap-south-1
```

Appendix D. Architecture References

The following AWS documentation pages were used to check the service behavior described in this document. They are external references, not project evidence.

- AWS Lambda asynchronous invocation retry behavior: https://docs.aws.amazon.com/lambda/latest/dg/invocation-async-error-handling.html
- AWS Lambda asynchronous invocation destinations and the requestPayload envelope: https://docs.aws.amazon.com/lambda/latest/dg/invocation-async-retain-records.html
- AWS Lambda deployment packages and ZIP limits: https://docs.aws.amazon.com/lambda/latest/dg/configuration-function-zip.html
- AWS Lambda runtimes and Python 3.13: https://docs.aws.amazon.com/lambda/latest/dg/lambda-python.html
- Amazon S3 versioning: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html
- Amazon S3 noncurrent version expiration: https://docs.aws.amazon.com/AmazonS3/latest/API/API_NoncurrentVersionExpiration.html
- Amazon S3 Block Public Access: https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html
- Amazon S3 server-side encryption: https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html

- Amazon S3 event notifications: https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html
- Amazon DynamoDB conditional writes: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html#WorkingWithItems.ConditionalUpdate
- Amazon SQS visibility timeout and retention: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
- AWS IAM least privilege: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html#grant-least-privilege
- Pillow documentation: https://pillow.readthedocs.io/

Appendix E. Screenshot Evidence Index

The table lists every figure used in the case study, its source capture, and what the capture actually supports. Original screenshots remain unchanged in the Images directory. Working copies in Assets/figures add a hairline border and, for five captures, crop the browser top bar or account menu; no content inside the console panels was removed.

Table E.1. *Figure, source, and evidence supported.*

Figure	Source capture	What it supports	Status
4.1	Original diagram	Architecture and data flow.	Original explanatory figure.
4.2	Original diagram	Per-version processing sequence.	Original explanatory figure.
4.3	Lambda 191128	S3 trigger, function, SQS destination, ARN.	Visually verified.
6.1	S3 190755	Bucket name and incoming/ prefix.	Visually verified.
6.2	S3 190819	Objects under incoming/.	Visually verified.
6.3	DynamoDB 190023	Table browser and Scan view.	Visually verified.
6.4	Lambda 191121	Function list, Zip package, Python 3.13.	Visually verified.
6.5	Lambda 191139	512 MB memory, 30-second timeout.	Visually verified.
6.6	Lambda 191134	Environment variables.	Visually verified.
6.7	Lambda 191151	ObjectCreated:*, incoming/ prefix, source account.	Visually verified.
6.8	CloudWatch 191323	Log group exists.	Visually verified.
6.9	CloudWatch 191344	Seven log streams.	Visually verified.
8.1	IAM 191509	Role page and ARN; policy document not shown.	Partially verified.
8.2	Lambda 191203	On-failure SQS destination.	Visually verified.
8.3	SQS 190605	One received message; receive count 1.	Visually verified.
9.1	S3 190827	Positive and negative test folders.	Visually verified.
9.2	S3 190834	Negative fixtures.	Visually verified.
9.3	S3 190841	Positive fixtures and nested/ folder.	Visually verified.
9.4	DynamoDB 190040	25-item scan and completed/failed rows.	Visually verified.
9.5	DynamoDB 190049	Additional rows and zero-byte records.	Visually verified.
9.6	DynamoDB 190058	Additional rows and failed fixtures.	Visually verified.
9.7	DynamoDB 190103	Final rows including nested image.	Visually verified.
9.8	CloudWatch 191404	Structured JSON lines and failed invocation.	Visually verified.
10.1	CloudWatch 191412	Two failed invocations and REPORT durations.	Visually verified.

Appendix F. Glossary

Table F.1. *Glossary of project terms.*

Term	Definition
Claim	The conditional PutItem that creates a PROCESSING record for an object version.
Conditional write	A DynamoDB write that succeeds only if a condition expression is true.
Content signature	The leading bytes of a file that identify a format, such as FF D8 FF for JPEG.
Deployment package	The ZIP archive uploaded to Lambda, containing handler.py, the processors, and runtime dependencies.
Failure destination	The Lambda asynchronous invocation destination that receives an invocation envelope after retries are exhausted.
Idempotency	The property that repeated delivery of the same event does not create a second logical result or corrupt an existing one.
manylinux_2_28_aarch64	A Python wheel platform tag for Linux arm64 with glibc 2.28 or newer.
Noncurrent version	An S3 object version that is no longer the latest version for its key.
Processing identity	The deterministic SHA-256 key derived from bucket, object key, and version ID.
Terminal state	COMPLETED or FAILED; the state after which no further transition is expected except a guarded re-attempt of FAILED.
Version-pinned read	A HeadObject or GetObject call that includes the event's version ID.